

# CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Technology and Engineering CSPIT-IT

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

| Practical 1: Introduction to React and Component Architecture |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CO/PO                                                         | CO1 / PO3, PO5                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Objective                                                     | To set up a React development environment using Vite and build a static UI using independently structured, reusable components.                                                                                                                                                                                                                                                                                                                                                                     |
| Industry Relevance                                            | Mirrors the standard local dev setup used in modern frontend teams<br>Vite-based scaffolding and component reuse are foundational skills expected from day one in any React role.                                                                                                                                                                                                                                                                                                                   |
| Prerequisites                                                 | <ul style="list-style-type: none"><li>• Basic HTML and CSS</li><li>• Familiarity with JavaScript fundamentals (variables, functions)</li><li>• Node.js and npm installed on the lab machine</li></ul>                                                                                                                                                                                                                                                                                               |
| Theory Concepts (Self-Study Reference)                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Concepts to Review                                            | <ul style="list-style-type: none"><li>• What React is and why component-based UI development is used over plain HTML/JS</li><li>• JSX syntax embedding JavaScript expressions inside markup</li><li>• Functional components and the role of the return statement</li><li>• Props passing data from a parent component into a child component</li><li>• Why reusable components reduce duplication and improve maintainability</li></ul>                                                             |
| Architecture / Diagram                                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Component Tree                                                | <pre>App.jsx ├── Header.jsx      (site title, name) ├── About.jsx       (short bio text) ├── Skills.jsx      (list of skills, receives props) └── Footer.jsx     (contact / copyright info)</pre>                                                                                                                                                                                                                                                                                                   |
| Problem Definition                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Problem Statement                                             | <ul style="list-style-type: none"><li>• Create a React application using Vite for a student portfolio page.</li><li>• The application must include at least 4 reusable components: Header, About, Skills, and Footer.</li><li>• Each component must be independently structured and composed into a single-page layout.</li><li>• No logic or JSX should be duplicated across components.</li><li>• Props must be used to pass at least one piece of data into a minimum of 2 components.</li></ul> |
| Key Questions / Analysis                                      | <ul style="list-style-type: none"><li>• What is the role of each component in the overall UI structure?</li><li>• How does React re-render when props change?</li><li>• Why is component reusability important in large-scale applications?</li></ul>                                                                                                                                                                                                                                               |
| Supplementary Problems                                        | <ul style="list-style-type: none"><li>• Add a NavBar component that highlights the active section.</li><li>• Pass an array of skills as a prop to the Skills component and render them dynamically.</li><li>• Add a theme color prop to the Header and apply it as an inline style.</li></ul>                                                                                                                                                                                                       |
| Key Skills Addressed                                          | <ul style="list-style-type: none"><li>• Setting up a React project using Vite</li><li>• Designing and structuring reusable functional components</li><li>• Passing data between components using props</li><li>• Composing a single-page layout from multiple components</li></ul>                                                                                                                                                                                                                  |

|                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |     |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| Learning Outcome      | Students will be able to set up a React development environment, create reusable components, and compose them into a working UI using props for data flow.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |     |
| Steps                 | Lab Session Step-by-Step                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |     |
|                       | <div>1. Verify Node.js and npm are installed:<br/>node -v<br/>npm -v</div> <div>2. Scaffold the React app using Vite:<br/>npm create vite@latest student-portfolio -- --template react<br/>cd student-portfolio<br/>npm install<br/>npm run dev</div> <div>3. Inside src/components, create four files: Header.jsx, About.jsx, Skills.jsx, Footer.jsx.</div> <div>4. Build the Skills component to accept a props array and render it as a list:<br/>function Skills({ skillList }) {<br/>  return (<br/>    &lt;ul&gt;<br/>      {skillList.map((s) =&gt; &lt;li key={s}&gt;{s}&lt;/li&gt;)}<br/>    &lt;/ul&gt;<br/>  );<br/>}</div> <div>5. Import and compose all four components inside App.jsx in a single layout.</div> <div>6. Pass a name prop to Header and a skillList prop to Skills from App.jsx.</div> <div>7. Run the app and verify all components render correctly without console errors.</div> |     |
|                       | <div>Tools / Technology / Resources Required</div> <ul style="list-style-type: none"><li>• Node.js (v18+), npm</li><li>• Visual Studio Code</li><li>• Vite, React 18+</li><li>• Browser DevTools (Console + React DevTools extension)</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |     |
|                       | <div>Total Hours</div> <ul style="list-style-type: none"><li>• Lab Session: 2 hours</li><li>• Implementation: 1.5 hours</li><li>• Testing / Modification: 0.5 hours</li><li>• Total Engagement: 2 hours</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |     |
|                       | Faculty Teaching Guide                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |     |
| Guidance Notes        | <ul style="list-style-type: none"><li>• Pre-check that lab machines have Node.js installed; keep an offline installer ready as backup.</li><li>• Live-demo the Vite scaffold once, slowly, before letting students repeat it independently.</li><li>• Walk the room during npm install slow lab Wi-Fi or proxy settings are the most common blocker.</li><li>• Stress 'one component, one responsibility' while reviewing student component structure.</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                  |     |
| Common Mistakes       | <ul style="list-style-type: none"><li>• Using an outdated Node version, causing the Vite scaffold command to fail.</li><li>• Putting all UI code inside App.jsx instead of splitting into separate components.</li><li>• Hardcoding values instead of passing them as props.</li><li>• Forgetting to save a file before checking the browser, leading to confusion about why changes aren't visible.</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |     |
| Troubleshooting Guide |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |     |
| Symptom               | Likely Cause                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | Fix |

|                                  |                                             |                                                                                     |
|----------------------------------|---------------------------------------------|-------------------------------------------------------------------------------------|
| npm create vite hangs            | Network/proxy blocked                       | Check lab internet connection; retry on stable network or use offline cache         |
| Port 5173 already in use         | Old dev server still running                | Stop the previous process or run<br>npm run dev -- --port 5174                      |
| Component not visible in browser | Component not imported/composed in App.jsx  | Confirm import statement and that the component is placed inside the return block   |
| Props show as undefined          | Prop name mismatch between parent and child | Verify the prop name passed in App.jsx matches the name used in the child component |

### Student Assignment / Post Laboratory Work

|                                     |                                                                                                                                                                                                                                                                                                                                               |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tasks</b>                        | <ul style="list-style-type: none"> <li>• Review the Theory Concepts section and the assigned Coursera module before next week.</li> <li>• Add a Projects component to the portfolio that renders a hardcoded list of 3 projects.</li> <li>• Experiment with passing different prop values and observe how the rendered UI changes.</li> </ul> |
| <b>GitHub Deliverables</b>          |                                                                                                                                                                                                                                                                                                                                               |
| <b>Expected Repository Contents</b> | <ul style="list-style-type: none"> <li>• Public repository named portfolio-&lt;rollno&gt;</li> <li>• Working Vite + React project with 4+ components</li> <li>• Clean .gitignore (node_modules excluded)</li> <li>• At least one meaningful commit message describing the work</li> </ul>                                                     |

### Rubrics (Practical Evaluation / Viva)

| Criteria (Marks)                       | Description                                                                                                                         |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>Environment Setup (3 marks)</b>     | All 4 tools (Node, npm, VS Code, Vite) verified via terminal; React app created and runs successfully using Vite without errors.    |
| <b>Project Structure (3 marks)</b>     | Components placed in a dedicated /components folder; no unused files; folder structure is clean and logical.                        |
| <b>Component Reusability (5 marks)</b> | Minimum 4 components created; each is independently structured; no JSX or logic duplicated across components.                       |
| <b>Props Usage (4 marks)</b>           | At least 2 components receive and render props correctly; no hardcoded values inside components that should be dynamic.             |
| <b>UI Output (5 marks)</b>             | App renders in browser without errors; all 4 components are visible; layout is coherent and components are clearly distinguishable. |
| <b>Total: 20 marks</b>                 | <b>Passing Threshold: 14 / 20 (70%)</b>                                                                                             |

| Practical 2: State Management and Routing in React |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CO/PO                                              | CO1 / PO3, PO5                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Objective                                          | To implement reactive state management using useState and multi-page navigation using React Router in the existing portfolio application.                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Prerequisites                                      | <ul style="list-style-type: none"> <li>• Practical 1 completed working portfolio app with reusable components</li> <li>• Basic understanding of JavaScript array/object handling</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                      |
| Theory Concepts (Self-Study Reference)             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Coursera Pointer                                   | See Week 2 IBM: Developing Front-End Apps with React, Module 2 (props, event handling, component composition, useState hook).                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Diagram                                            | Architecture / Diagram                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|                                                    | <pre> App.jsx (wrapped in &lt;BrowserRouter&gt;) ├── NavBar.jsx           (links to all routes) ├── Route: "/"           → Home.jsx ├── Route: "/projects"   → Projects.jsx └── Route: "/contact"    → Contact.jsx (controlled form,                            useState) </pre>                                                                                                                                                                                                                                                                                                 |
| Problem Definition                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Problem Statement                                  | <ul style="list-style-type: none"> <li>• Extend the portfolio application from Practical 1 by adding React Router.</li> <li>• Implement a navigation bar with at least 3 routes: Home, Projects, and Contact.</li> <li>• Each route must render a distinct component without a full page reload.</li> <li>• Add at least 2 useState variables used meaningfully one for toggling UI visibility and one for managing a form input on the Contact page.</li> <li>• The Contact page must include a controlled input that captures and displays user input in real time.</li> </ul> |
| Key Questions / Analysis                           | <ul style="list-style-type: none"> <li>• What is the role of each component in the overall UI structure?</li> <li>• How does React re-render when props change?</li> <li>• Why is component reusability important in large-scale applications?</li> </ul>                                                                                                                                                                                                                                                                                                                        |
| Supplementary Problems                             | <ul style="list-style-type: none"> <li>• Add a 404 Not Found route that renders a custom error component for unknown paths.</li> <li>• Implement a dark/light mode toggle using useState that applies a CSS class to the root element.</li> <li>• Store the contact form input in state and display a live character count below the input.</li> </ul>                                                                                                                                                                                                                           |
| Key Skills Addressed                               | <ul style="list-style-type: none"> <li>• Installing and configuring React Router v6</li> <li>• Defining and navigating between routes without page reload</li> <li>• Managing component state using the useState hook</li> <li>• Building controlled form inputs tied to state</li> </ul>                                                                                                                                                                                                                                                                                        |
| Learning Outcome                                   | Students will be able to implement client-side routing with React Router and manage dynamic UI state using useState, resulting in a multi-page application without full page reloads.                                                                                                                                                                                                                                                                                                                                                                                            |
| Lab Session Step-by-Step                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Steps                                              | <ol style="list-style-type: none"> <li>1. Install React Router in the existing portfolio project:<br/> <pre>npm install react-router-dom</pre> </li> <li>2. Wrap the App component with BrowserRouter in main.jsx:<br/> <pre>import { BrowserRouter } from 'react-router-dom'; &lt;BrowserRouter&gt;   &lt;App /&gt; &lt;/BrowserRouter&gt;</pre> </li> <li>3. Define 3 routes inside App.jsx using Routes and Route:<br/> <pre>&lt;Routes&gt;</pre> </li> </ol>                                                                                                                 |

|                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
|                                           | <pre>&lt;Route path="/" element={&lt;Home /&gt;} /&gt; &lt;Route path="/projects" element={&lt;Projects /&gt;} /&gt; &lt;Route path="/contact" element={&lt;Contact /&gt;} /&gt; &lt;/Routes&gt;</pre> <p>4. Build the NavBar using Link (not &lt;a&gt; tags) to avoid full page reloads.</p> <p>5. Inside Contact.jsx, create a controlled input tied to useState:</p> <pre>const [message, setMessage] = useState(''); &lt;input value={message} onChange={(e) =&gt; setMessage(e.target.value)} /&gt;</pre> <p>6. Add a second useState variable to toggle visibility of an element (e.g., a help tooltip).</p> <p>7. Test navigation across all 3 routes and confirm no page reload occurs.</p> |                                                                                |
| Tools / Technology / Resources Required   | <ul style="list-style-type: none"><li>• Node.js (v18+), npm</li><li>• Visual Studio Code</li><li>• React 18+, React Router v6</li><li>• Browser DevTools</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |                                                                                |
| Total Hours                               | <ul style="list-style-type: none"><li>• Lab Session: 2 hours</li><li>• Implementation: 1.5 hours</li><li>• Testing / Modification: 0.5 hours</li><li>• Total Engagement: 2 hours</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                |
| Faculty Teaching Guide                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                |
| Guidance Notes                            | <ul style="list-style-type: none"><li>• Emphasize Link vs &lt;a&gt; this is the most common point of confusion for students new to SPA routing.</li><li>• Live-demo useState updating the UI in real time before students attempt the Contact form.</li><li>• Check that BrowserRouter wraps the App only once, typically in main.jsx, not repeated elsewhere.</li><li>• Walk through how routes map to components before students start writing code.</li></ul>                                                                                                                                                                                                                                    |                                                                                |
| Common Mistakes                           | <ul style="list-style-type: none"><li>• Using &lt;a href="..."&gt; instead of &lt;Link to="..."&gt;, causing full page reloads.</li><li>• Forgetting to wrap the app in BrowserRouter, resulting in a 'useRoutes() may be used only in the context of a Router' error.</li><li>• Using props instead of state for values that should change dynamically (like form input).</li><li>• Not providing a unique path for each Route, causing routes to render incorrectly.</li></ul>                                                                                                                                                                                                                    |                                                                                |
| Troubleshooting Guide                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                |
| Symptom                                   | Likely Cause                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Fix                                                                            |
| Blank page after adding routes            | BrowserRouter not wrapping App component                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | Confirm BrowserRouter wraps <App /> in main.jsx                                |
| Clicking nav link reloads page            | Used <a> tag instead of <Link>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | Replace all navigation anchors with <Link to="..."> from react-router-dom      |
| Form input not updating on typing         | Missing onChange handler or value not tied to state                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | Ensure input has both value={state} and onChange={set state}                   |
| Route shows wrong component               | Path mismatch or routes defined in wrong order                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | Double check path strings match exactly; order routes from specific to general |
| Student Assignment / Post Laboratory Work |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                |
| Tasks                                     | <ul style="list-style-type: none"><li>• Review the Theory Concepts section and the assigned Coursera module before next week.</li><li>• Add a 404 Not Found route for any undefined path.</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                |

|                                              | <ul style="list-style-type: none"> <li>Implement a dark/light mode toggle using a second useState variable.</li> </ul>                                                                                                                                                                                |
|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>GitHub Deliverables</b>                   |                                                                                                                                                                                                                                                                                                       |
| <b>Expected Repository Contents</b>          | <ul style="list-style-type: none"> <li>Same repository as Practical 1, updated with new commits</li> <li>Working multi-route app with NavBar, Home, Projects, Contact</li> <li>At least one commit specifically for routing implementation</li> <li>Updated README noting new routes added</li> </ul> |
| <b>Rubrics (Practical Evaluation / Viva)</b> |                                                                                                                                                                                                                                                                                                       |
| Criteria (Marks)                             | Description                                                                                                                                                                                                                                                                                           |
| <b>React Router Setup (4 marks)</b>          | React Router v6 installed; BrowserRouter wraps the app correctly; no console routing errors on load.                                                                                                                                                                                                  |
| <b>Route Configuration (4 marks)</b>         | Minimum 3 routes defined; each route renders the correct component; navigation links work without triggering a page reload.                                                                                                                                                                           |
| <b>useState UI Toggle (4 marks)</b>          | At least one UI element toggles visibility or appearance based on a state variable; state change is reflected immediately in the UI.                                                                                                                                                                  |
| <b>Controlled Form Input (5 marks)</b>       | Contact page form input is a controlled component tied to state via value and onChange; input value updates correctly on every keystroke.                                                                                                                                                             |
| <b>Navigation UX (3 marks)</b>               | Active route is visually distinguishable in the NavBar; navigating between all 3 routes works correctly without errors.                                                                                                                                                                               |
| <b>Total: 20 marks</b>                       | <b>Passing Threshold: 14 / 20 (70%)</b>                                                                                                                                                                                                                                                               |

| API Integration and Data Rendering in React   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CO/PO                                         | CO1 / PO3, PO5                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Objective                                     | To consume a REST API in React and handle asynchronous data with loading and error states.                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Prerequisites                                 | <ul style="list-style-type: none"> <li>Practical 1 and 2 completed multi-route portfolio app with state management</li> <li>Basic understanding of JavaScript Promises and async/await</li> </ul>                                                                                                                                                                                                                                                                                                                              |
| <b>Theory Concepts (Self-Study Reference)</b> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Coursera Pointer                              | See Week 3 IBM: Developing Front-End Apps with React, Module 3 (React hooks, form management, connecting to external servers, async data fetching).                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Architecture / Diagram</b>                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Diagram                                       | <pre> Projects.jsx ├── useEffect() → triggers fetch on mount ├── useState: data, loading, error ├── [loading] → &lt;Spinner /&gt; ├── [error] → &lt;ErrorMessage /&gt; └── [success] → &lt;RepoList data={data} /&gt; </pre>                                                                                                                                                                                                                                                                                                   |
| <b>Problem Definition</b>                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Problem Statement                             | <ul style="list-style-type: none"> <li>Integrate a public REST API (e.g., GitHub API) into the portfolio application to fetch and display a list of repositories dynamically.</li> <li>Use the Projects page built in Practical 2 as the integration point.</li> <li>Implement a loading spinner component shown while the request is in progress.</li> <li>Implement an error message component shown if the API call fails.</li> <li>Render at least the repository name and URL for each item returned.</li> </ul>          |
| Key Questions / Analysis                      | <ul style="list-style-type: none"> <li>Why is useEffect required to trigger a fetch on component mount instead of calling fetch directly in the component body?</li> <li>What is the difference between a loading state and an error state, and why must both be handled separately?</li> <li>How would the user experience change if loading and error states were not implemented?</li> </ul>                                                                                                                                |
| Supplementary Problems                        | <ul style="list-style-type: none"> <li>Add a retry button that re-triggers the fetch when an error occurs.</li> <li>Add a search input that filters the rendered repository list by name.</li> <li>Display the repository's star count alongside its name.</li> </ul>                                                                                                                                                                                                                                                          |
| Key Skills Addressed                          | <ul style="list-style-type: none"> <li>Fetching data from a REST API using fetch or axios</li> <li>Using useEffect to trigger side effects on component mount</li> <li>Managing asynchronous state loading, success, and error</li> <li>Conditionally rendering UI based on request state</li> </ul>                                                                                                                                                                                                                           |
| Learning Outcome                              | Students will be able to consume a REST API in React, manage asynchronous state correctly, and provide appropriate UI feedback during all stages of a network request.                                                                                                                                                                                                                                                                                                                                                         |
| <b>Lab Session Step-by-Step</b>               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Steps                                         | <ol style="list-style-type: none"> <li>Inside Projects.jsx, set up 3 state variables for data, loading, and error: <pre> const [repos, setRepos] = useState([]); const [loading, setLoading] = useState(true); const [error, setError] = useState(null); </pre> </li> <li>Use useEffect to fetch data when the component mounts: <pre> useEffect(() =&gt; {   fetch('https://api.github.com/users/&lt;username&gt;/repos')     .then((res) =&gt; res.json())     .then((data) =&gt; setRepos(data)) }, []); </pre> </li> </ol> |

|                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                |
|----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
|                                                    | <pre>        .catch((err) =&gt; setError(err.message))         .finally(() =&gt; setLoading(false));     }, []);</pre> <p>3. Conditionally render based on state:<br/>if (loading) return &lt;Spinner /&gt;;<br/>if (error) return &lt;ErrorMessage message={error} /&gt;;</p> <p>4. Map over the repos array and render name and html_url for each repository.</p> <p>5. Test the happy path confirm data renders correctly after loading.</p> <p>6. Test the error path temporarily break the API URL and confirm the error message displays.</p>             |                                                                                                |
| Tools / Technology / Resources Required            | <ul style="list-style-type: none"><li>• Node.js (v18+), npm</li><li>• Visual Studio Code</li><li>• React 18+, Fetch API or Axios</li><li>• Browser DevTools (Network tab)</li></ul>                                                                                                                                                                                                                                                                                                                                                                             |                                                                                                |
| Total Hours                                        | <ul style="list-style-type: none"><li>• Lab Session: 2 hours</li><li>• Implementation: 1.5 hours</li><li>• Testing / Modification: 0.5 hours</li><li>• Total Engagement: 2 hours</li></ul>                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                |
| Guidance Notes                                     | <h3>Faculty Teaching Guide</h3> <ul style="list-style-type: none"><li>• Use a public API requiring no authentication (GitHub API is ideal) to avoid API key issues in lab.</li><li>• Demonstrate breaking the API URL deliberately to show the error state students rarely test this on their own.</li><li>• Explain why the dependency array [] in useEffect matters fetching on every render is a very common bug to anticipate.</li><li>• Walk through the difference between .then() chains and async/await if students choose to use the latter.</li></ul> |                                                                                                |
|                                                    | <ul style="list-style-type: none"><li>• Calling fetch directly inside the component body without useEffect, causing infinite re-fetching.</li><li>• Omitting the dependency array in useEffect, causing the request to fire on every render.</li><li>• Not handling the error case at all, causing the app to crash or show a blank screen on failure.</li><li>• Forgetting to set loading back to false after the fetch completes, leaving the spinner stuck.</li></ul>                                                                                        |                                                                                                |
| Common Mistakes                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                |
| <h3>Troubleshooting Guide</h3>                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                |
| Symptom                                            | Likely Cause                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Fix                                                                                            |
| Infinite network requests in console               | Missing dependency array in useEffect                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | Add an empty array [] as the second argument to useEffect                                      |
| Spinner never disappears                           | setLoading(false) not called in .finally() or after both success/error paths                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Use .finally() to guarantee loading is set to false regardless of outcome                      |
| App crashes on API failure                         | No error boundary or conditional check before rendering data                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Add error state handling and conditionally render ErrorMessage before the data list            |
| CORS error in console                              | API does not allow cross-origin requests from localhost                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | Use a CORS-friendly public API like GitHub's; do not attempt to bypass CORS with browser flags |
| <h3>Student Assignment / Post Laboratory Work</h3> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                |
| Tasks                                              | <ul style="list-style-type: none"><li>• Review the Theory Concepts section and the assigned Coursera module before next week.</li><li>• Add a retry button that re-triggers the fetch when an error occurs.</li><li>• Add a simple search/filter input above the repository list.</li></ul>                                                                                                                                                                                                                                                                     |                                                                                                |

| GitHub Deliverables                   |                                                                                                                                                                                                                                                                                                                                            |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Expected Repository Contents          | <ul style="list-style-type: none"> <li>• Same repository as Practical 1 and 2, updated with new commits</li> <li>• Working API integration on the Projects page with loading and error states</li> <li>• At least one commit specifically for API integration</li> <li>• Updated README noting the API used and any setup steps</li> </ul> |
| Rubrics (Practical Evaluation / Viva) |                                                                                                                                                                                                                                                                                                                                            |
| Criteria (Marks)                      | Description                                                                                                                                                                                                                                                                                                                                |
| API Call Implementation (5 marks)     | fetch or axios used correctly; API URL is valid; request fires on component mount using useEffect.                                                                                                                                                                                                                                         |
| Data Rendering (5 marks)              | API response parsed and mapped correctly; list of repositories displayed with at least name and URL.                                                                                                                                                                                                                                       |
| Loading State (4 marks)               | Loading spinner/indicator visible while data is being fetched; disappears after data loads.                                                                                                                                                                                                                                                |
| Error Handling (4 marks)              | Error message component renders on failed API call; does not crash the app.                                                                                                                                                                                                                                                                |
| Code Cleanliness (2 marks)            | No console.log left in code; async logic separated from JSX.                                                                                                                                                                                                                                                                               |
| Total: 20 marks                       | Passing Threshold: 14 / 20 (70%)                                                                                                                                                                                                                                                                                                           |

| Practical 4: Building a RESTful API with Node.js and Express |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CO/PO                                                        | CO2 / PO3, PO5                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Objective                                                    | To design and implement a RESTful backend server with complete CRUD endpoints using Express middleware pipeline.                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Prerequisites                                                | <ul style="list-style-type: none"> <li>• Basic JavaScript (functions, objects, arrays, async basics)</li> <li>• Understanding of HTTP methods (GET, POST, PUT, DELETE) and status codes</li> </ul>                                                                                                                                                                                                                                                                                                                                     |
| Coursera Pointer                                             | <b>Theory Concepts (Self-Study Reference)</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|                                                              | See Week 4 IBM: Node.js & MongoDB Developing Back-end Database Applications, Module 1 (RESTful API design with Node.js and Express, middleware pipeline, request lifecycle).                                                                                                                                                                                                                                                                                                                                                           |
| Architecture / Diagram                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Diagram                                                      | <pre> Client (Postman/Browser)       v [Logging Middleware]       v Express Router ├── GET    /tasks      → getAllTasks ├── POST   /tasks      → createTask ├── PUT    /tasks/:id  → updateTask ├── DELETE /tasks/:id  → deleteTask       v [Global Error Handler] </pre>                                                                                                                                                                                                                                                              |
| Problem Definition                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Problem Statement                                            | <ul style="list-style-type: none"> <li>• Build a Node/Express backend for a Task Management system.</li> <li>• Implement REST endpoints for creating, reading, updating, and deleting tasks (in-memory array, no database yet).</li> <li>• Apply a request logging middleware that logs method, URL, and timestamp for every incoming request.</li> <li>• Apply a global error handling middleware as the last middleware in the pipeline.</li> <li>• Use correct HTTP status codes (200, 201, 404, 500) for each response.</li> </ul> |
| Key Questions / Analysis                                     | <ul style="list-style-type: none"> <li>• Why must the error handling middleware be defined last in the middleware chain?</li> <li>• What is the difference between <code>app.use()</code> and a route-specific middleware?</li> <li>• Why is it considered bad practice to send raw error stack traces to the client?</li> </ul>                                                                                                                                                                                                       |
| Supplementary Problems                                       | <ul style="list-style-type: none"> <li>• Add a middleware that rejects requests without a Content-Type: application/json header on POST/PUT.</li> <li>• Add a route-specific middleware that validates the task ID format before reaching the controller.</li> <li>• Add a 404 handler for undefined routes that returns a structured JSON response.</li> </ul>                                                                                                                                                                        |
| Key Skills Addressed                                         | <ul style="list-style-type: none"> <li>• Setting up an Express server and defining RESTful routes</li> <li>• Writing and applying custom middleware functions</li> <li>• Structuring a request-response-error pipeline correctly</li> <li>• Returning appropriate HTTP status codes for each operation</li> </ul>                                                                                                                                                                                                                      |

|                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |     |  |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|--|
| Learning Outcome                        | Students will be able to design and implement a complete RESTful API using Express, including a working middleware pipeline with logging and centralized error handling.                                                                                                                                                                                                                                                                                                            |     |  |
| Steps                                   | Lab Session Step-by-Step                                                                                                                                                                                                                                                                                                                                                                                                                                                            |     |  |
|                                         | 1. Initialize a new Node project and install Express:<br>mkdir task-manager-api && cd task-manager-api<br>npm init -y<br>npm install express                                                                                                                                                                                                                                                                                                                                        |     |  |
|                                         | 2. Create server.js and set up a basic Express app:<br>const express = require('express');<br>const app = express();<br>app.use(express.json());<br>app.listen(5000, () => console.log('Server running on port 5000'));                                                                                                                                                                                                                                                             |     |  |
|                                         | 3. Add a request logging middleware applied globally:<br>app.use((req, res, next) => {<br>console.log(`\${req.method} \${req.url} - \${new Date().toISOString()}`);<br>next();<br>});                                                                                                                                                                                                                                                                                               |     |  |
|                                         | 4. Implement the 4 CRUD routes using an in-memory array as temporary storage.                                                                                                                                                                                                                                                                                                                                                                                                       |     |  |
| Tools / Technology / Resources Required | 5. Add a global error handling middleware as the final middleware:<br>app.use((err, req, res, next) => {<br>console.error(err.stack);<br>res.status(500).json({ error: 'Something went wrong' });<br>});                                                                                                                                                                                                                                                                            |     |  |
|                                         | 6. Test all 4 endpoints using Postman or Thunder Client, verifying correct status codes.                                                                                                                                                                                                                                                                                                                                                                                            |     |  |
| Total Hours                             | <ul style="list-style-type: none"><li>• Node.js (v18+), npm</li><li>• Express.js</li><li>• Postman or Thunder Client</li><li>• Visual Studio Code</li></ul>                                                                                                                                                                                                                                                                                                                         |     |  |
| Guidance Notes                          | <ul style="list-style-type: none"><li>• Lab Session: 2 hours</li><li>• Implementation: 1.5 hours</li><li>• Testing / Modification: 0.5 hours</li><li>• Total Engagement: 2 hours</li></ul>                                                                                                                                                                                                                                                                                          |     |  |
|                                         | Faculty Teaching Guide                                                                                                                                                                                                                                                                                                                                                                                                                                                              |     |  |
| Common Mistakes                         | <ul style="list-style-type: none"><li>• Demonstrate the request lifecycle on the whiteboard/diagram before coding students often treat middleware as 'magic'.</li><li>• Emphasize calling next() in every middleware that doesn't end the response, or requests will hang indefinitely.</li><li>• Show a live example of the error handler catching a deliberately thrown error.</li><li>• Walk through Postman setup once as a class before students work independently.</li></ul> |     |  |
|                                         | <ul style="list-style-type: none"><li>• Forgetting to call next() inside custom middleware, causing the request to hang.</li><li>• Placing the error handling middleware before the routes instead of after all of them.</li><li>• Not setting app.use(express.json()), causing req.body to be undefined on POST/PUT.</li><li>• Returning inconsistent response formats across different endpoints.</li></ul>                                                                       |     |  |
| Troubleshooting Guide                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |     |  |
| Symptom                                 | Likely Cause                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Fix |  |

|                                   |                                                                         |                                                                                     |
|-----------------------------------|-------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Request hangs indefinitely        | Middleware does not call next() or send a response                      | Ensure every middleware either calls next() or ends the response with res.send/json |
| req.body is undefined             | Missing express.json() middleware                                       | Add app.use(express.json()) before route definitions                                |
| Error handler never triggers      | Error handler defined before routes, or errors not passed via next(err) | Move error handler to be the last app.use(); call next(err) inside try/catch blocks |
| Postman shows 'Cannot GET /tasks' | Route path mismatch or server not restarted after changes               | Verify route path spelling and restart the Node server                              |

### Student Assignment / Post Laboratory Work

#### Tasks

- Review the Theory Concepts section and the assigned Coursera module before next week.
- Add a middleware that rejects POST/PUT requests missing a Content-Type header.
- Add a custom 404 handler for undefined routes.

### GitHub Deliverables

#### Expected Repository Contents

- New repository named task-manager-api-<rollno>
- Working Express server with all 4 CRUD routes and middleware pipeline
- Clean .gitignore (node\_modules excluded)
- At least one meaningful commit message describing the work

### Rubrics (Practical Evaluation / Viva)

| Criteria (Marks)                            | Description                                                                                                           |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Server Setup (3 marks)</b>               | Express server runs on defined port; confirmed via terminal and browser/Postman.                                      |
| <b>REST Endpoints (8 marks)</b>             | All 4 CRUD endpoints (GET, POST, PUT, DELETE) implemented; correct HTTP methods and status codes used.                |
| <b>Request Logging Middleware (4 marks)</b> | Custom middleware logs method, URL, and timestamp for every request; applied globally.                                |
| <b>Global Error Handling (5 marks)</b>      | Error handling middleware defined last in pipeline; catches unhandled errors; returns structured JSON error response. |
| <b>Total: 20 marks</b>                      | <b>Passing Threshold: 14 / 20 (70%)</b>                                                                               |

| Practical 5: MongoDB Integration and Schema Design with Mongoose |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CO/PO                                                            | CO2, CO3 / PO3, PO5                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Objective                                                        | To connect a MongoDB database to an Express server and enforce data validation through Mongoose schema.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Prerequisites                                                    | <ul style="list-style-type: none"> <li>• Practical 4 completed working Express server with in-memory CRUD</li> <li>• Basic understanding of NoSQL/document-based data (vs relational tables)</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                      |
| Coursera Pointer                                                 | <b>Theory Concepts (Self-Study Reference)</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|                                                                  | See Week 5 IBM: Node.js & MongoDB Developing Back-end Database Applications, Module 2 (data and databases intro, MongoDB CRUD operations, Mongoose schema design and validation).                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Architecture / Diagram</b>                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Diagram                                                          | <pre> Express App       v Mongoose ODM       v MongoDB Database   └─ tasks collection       { title, description, completed, createdAt } </pre> <p>Schema Validation: required fields + default values enforced before any document reaches the database.</p>                                                                                                                                                                                                                                                                                                                                                |
| <b>Problem Definition</b>                                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Problem Statement                                                | <ul style="list-style-type: none"> <li>• Extend the Task Management backend from Practical 4 by connecting MongoDB using Mongoose.</li> <li>• Define a Task schema with at least 4 fields: title (String, required), description (String), completed (Boolean, default false), createdAt (Date, default Date.now).</li> <li>• Replace the in-memory array from Practical 4 with real Mongoose model operations.</li> <li>• Test all CRUD operations against the live database using Postman.</li> <li>• Ensure validation errors are returned as structured JSON, not raw Mongoose error objects.</li> </ul> |
| Key Questions / Analysis                                         | <ul style="list-style-type: none"> <li>• What is the purpose of a schema in a NoSQL database like MongoDB, given that MongoDB itself is schema-less?</li> <li>• Why is it important to define required fields and default values at the schema level rather than relying on frontend validation alone?</li> <li>• What happens internally when a document fails Mongoose validation where is the request stopped?</li> </ul>                                                                                                                                                                                 |
| Supplementary Problems                                           | <ul style="list-style-type: none"> <li>• Add a priority field to the schema restricted to an enum of ['low', 'medium', 'high'].</li> <li>• Add a pre-save hook that automatically trims whitespace from the title field.</li> <li>• Implement a GET /tasks/:id endpoint that returns a 404 JSON response if the ID does not exist.</li> </ul>                                                                                                                                                                                                                                                                |
| Key Skills Addressed                                             | <ul style="list-style-type: none"> <li>• Connecting a Node/Express app to MongoDB using Mongoose</li> <li>• Designing a schema with field types, required constraints, and defaults</li> <li>• Performing CRUD operations using Mongoose model methods</li> <li>• Returning structured validation error responses</li> </ul>                                                                                                                                                                                                                                                                                 |
| Learning Outcome                                                 | Students will be able to connect an Express application to MongoDB using Mongoose, define a validated schema, and perform real CRUD operations against a live database.                                                                                                                                                                                                                                                                                                                                                                                                                                      |

| Lab Session Step-by-Step                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                                                    |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Steps                                   | <div>1. Install Mongoose and create a .env file for the connection string:<br/>npm install mongoose dotenv</div> <div>2. Connect to MongoDB inside server.js:<br/>const mongoose = require('mongoose');<br/>require('dotenv').config();<br/>mongoose.connect(process.env.MONGO_URI)<br/>  .then(() =&gt; console.log('MongoDB connected'))<br/>  .catch((err) =&gt; console.error(err));</div> <div>3. Define the Task schema and model in a separate models/Task.js file:<br/>const taskSchema = new mongoose.Schema({<br/>  title: { type: String, required: true },<br/>  description: { type: String },<br/>  completed: { type: Boolean, default: false },<br/>  createdAt: { type: Date, default: Date.now }<br/>});<br/>module.exports = mongoose.model('Task', taskSchema);</div> <div>4. Replace the in-memory array logic from Practical 4 with Mongoose methods: Task.find(), Task.create(), Task.findByIdAndUpdate(), Task.findByIdAndDelete().</div> <div>5. Wrap each route in a try/catch and pass errors to the global error handler using next(err).</div> <div>6. Test all 4 endpoints in Postman; confirm data persists in MongoDB after server restart.</div> |                                                                                                                    |
| Tools / Technology / Resources Required | <ul style="list-style-type: none"><li>• Node.js (v18+), npm</li><li>• Express.js, Mongoose</li><li>• MongoDB (local or Atlas)</li><li>• Postman or Thunder Client</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                    |
| Total Hours                             | <ul style="list-style-type: none"><li>• Lab Session: 2 hours</li><li>• Implementation: 1.5 hours</li><li>• Testing / Modification: 0.5 hours</li><li>• Total Engagement: 2 hours</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                                                                                                                    |
| Faculty Teaching Guide                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                                                    |
| Guidance Notes                          | <ul style="list-style-type: none"><li>• Decide in advance: local MongoDB install vs MongoDB Atlas free tier demonstrate connection string setup for whichever is chosen.</li><li>• Show the MongoDB Compass GUI (or Atlas web UI) so students can visually confirm documents are being created.</li><li>• Emphasize that schema validation happens at the Mongoose layer, not in the database itself important for understanding NoSQL flexibility vs application-level enforcement.</li><li>• Walk through one validation failure live so students see the actual error shape before they handle it.</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                                                    |
| Common Mistakes                         | <ul style="list-style-type: none"><li>• Hardcoding the MongoDB connection string directly in code instead of using a .env file.</li><li>• Forgetting required: true on fields that should never be empty, allowing invalid documents to be saved.</li><li>• Not wrapping database calls in try/catch, causing unhandled promise rejections to crash the server.</li><li>• Sending the raw Mongoose ValidationError object to the client instead of a clean, structured message.</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |                                                                                                                    |
| Troubleshooting Guide                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                                                    |
| Symptom                                 | Likely Cause                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | Fix                                                                                                                |
| MongoDB connection times out            | Incorrect connection string or IP not whitelisted (Atlas)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | Verify MONGO_URI in .env; if using Atlas, whitelist the lab's IP address or allow access from anywhere for lab use |

|                                   |                                                       |                                                                                                |
|-----------------------------------|-------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Validation error not caught       | Mongoose operation not wrapped in try/catch           | Wrap all async Mongoose calls in try/catch and forward errors using next(err)                  |
| Data not persisting after restart | Still using in-memory array instead of Mongoose model | Confirm all CRUD routes use Task.find/create/update/delete, not the old array                  |
| Cannot find module 'dotenv'       | Package not installed                                 | Run npm install dotenv and ensure require('dotenv').config() is called at the top of server.js |

### Student Assignment / Post Laboratory Work

|                                     |                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tasks</b>                        | <ul style="list-style-type: none"> <li>Review the Theory Concepts section and the assigned Coursera module before next week.</li> <li>Add a priority field to the schema restricted to an enum of low/medium/high.</li> <li>Implement a GET /tasks/:id endpoint with proper 404 handling.</li> </ul>                                                                      |
| <b>Expected Repository Contents</b> | <p><b>GitHub Deliverables</b></p> <ul style="list-style-type: none"> <li>Same repository as Practical 4, updated with new commits</li> <li>Working MongoDB-backed CRUD API with Mongoose schema and validation</li> <li>.env excluded via .gitignore; .env.example provided instead</li> <li>At least one commit specifically for MongoDB/Mongoose integration</li> </ul> |

### Rubrics (Practical Evaluation / Viva)

| Criteria (Marks)                        | Description                                                                                                                                 |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <b>MongoDB Connection (3 marks)</b>     | Mongoose connects to MongoDB successfully; connection string in .env file; connection status logged.                                        |
| <b>Schema Design (5 marks)</b>          | Task schema has minimum 4 fields; correct data types assigned; at least 2 required constraints and 1 default value defined.                 |
| <b>CRUD via Mongoose (8 marks)</b>      | All 4 operations tested and working against live MongoDB; correct Mongoose methods used (save, find, findByIdAndUpdate, findByIdAndDelete). |
| <b>Validation Enforcement (4 marks)</b> | Missing required fields return validation error; error message is descriptive and returned as JSON.                                         |
| <b>Total: 20 marks</b>                  | <b>Passing Threshold: 14 / 20 (70%)</b>                                                                                                     |

| Practical 6: Full Stack Integration React + Node + MongoDB |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CO/PO                                                      | CO1, CO2 / PO3, PO5                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Objective                                                  | To wire the React frontend to the Node/Express/MongoDB backend into a fully functional full-stack application.                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Prerequisites                                              | <ul style="list-style-type: none"> <li>• Practicals 1–3 completed working React UI with routing and API consumption patterns</li> <li>• Practicals 4–5 completed working Node/Express/MongoDB backend with CRUD</li> </ul>                                                                                                                                                                                                                                                                                                                                                               |
|                                                            | <b>Theory Concepts (Self-Study Reference)</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Coursera Pointer                                           | See Week 6 IBM: Node.js & MongoDB Developing Back-end Database Applications, Module 3 (connecting frontend to backend, end-to-end API calls, full-stack flow).                                                                                                                                                                                                                                                                                                                                                                                                                           |
|                                                            | <b>Architecture / Diagram</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Diagram                                                    | <pre> React Frontend (localhost:5173)     fetch/axios calls   v Express Backend (localhost:5000)     Mongoose   v MongoDB Database </pre> <p>Flow: UI Form → POST /tasks → MongoDB → UI re-fetches → updated list rendered</p>                                                                                                                                                                                                                                                                                                                                                           |
|                                                            | <b>Problem Definition</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Problem Statement                                          | <ul style="list-style-type: none"> <li>• Connect the React Task UI to the Node+MongoDB backend built in Practicals 4 and 5.</li> <li>• Replace the GitHub repo data from Practical 3 with task data from your own backend.</li> <li>• The application must support creating, viewing, updating, and deleting tasks from the React interface through API calls.</li> <li>• All data must be persisted in MongoDB confirmed by refreshing the browser and seeing data remain.</li> <li>• Handle loading and error states for every API interaction, not just the initial fetch.</li> </ul> |
| Key Questions / Analysis                                   | <ul style="list-style-type: none"> <li>• What changes are required on the backend (CORS) to allow the React dev server to call the Express API?</li> <li>• Why should the UI re-fetch or update local state after a successful POST/PUT/DELETE rather than assuming success silently?</li> <li>• What is the risk of not handling errors on write operations (POST/PUT/DELETE) the same way as read operations (GET)?</li> </ul>                                                                                                                                                         |
| Supplementary Problems                                     | <ul style="list-style-type: none"> <li>• Add an optimistic UI update for task creation show the new task in the list immediately, before the server confirms.</li> <li>• Add a confirmation dialog before deleting a task.</li> <li>• Add a toast/notification component that shows success or failure after each operation.</li> </ul>                                                                                                                                                                                                                                                  |
| Key Skills Addressed                                       | <ul style="list-style-type: none"> <li>• Configuring CORS on an Express server for local frontend-backend communication</li> <li>• Calling backend CRUD endpoints from React using fetch or axios</li> <li>• Synchronizing UI state with server state after write operations</li> <li>• Handling loading and error states across multiple API interactions in one component</li> </ul>                                                                                                                                                                                                   |

|                                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                       |
|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| <b>Learning Outcome</b>                        | Students will be able to integrate a React frontend with a Node/Express/MongoDB backend into a complete, working full-stack application with proper state synchronization.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                                                                       |
| <b>Steps</b>                                   | <b>Lab Session Step-by-Step</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                       |
|                                                | <ol style="list-style-type: none"> <li>1. Install and configure CORS on the Express backend:<br/> <pre>npm install cors const cors = require('cors'); app.use(cors());</pre> </li> <li>2. In React, create a central api.js file with the base backend URL:<br/> <pre>const BASE_URL = 'http://localhost:5000'; export const getTasks = () =&gt; fetch(`\${BASE_URL}/tasks`).then(res =&gt; res.json());</pre> </li> <li>3. Replace the Practical 3 GitHub fetch logic with calls to your own /tasks endpoint.</li> <li>4. Build a task creation form that POSTs to /tasks and updates local state on success.</li> <li>5. Implement update (PUT) and delete (DELETE) actions, each followed by a state update or re-fetch.</li> <li>6. Wrap each API call in proper loading/error handling, reusing the pattern from Practical 3.</li> <li>7. Test the complete flow: create → view → update → delete, refreshing the browser to confirm persistence.</li> </ol> |                                                                       |
| <b>Tools / Technology / Resources Required</b> | <ul style="list-style-type: none"> <li>• Node.js (v18+), npm</li> <li>• React 18+, Express.js, Mongoose</li> <li>• cors package</li> <li>• Postman (for backend verification), Browser DevTools</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                                                                       |
| <b>Total Hours</b>                             | <ul style="list-style-type: none"> <li>• Lab Session: 2 hours</li> <li>• Implementation: 1.5 hours</li> <li>• Testing / Modification: 0.5 hours</li> <li>• Total Engagement: 2 hours</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                       |
| <b>Guidance Notes</b>                          | <b>Faculty Teaching Guide</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                       |
|                                                | <ul style="list-style-type: none"> <li>• This is the first practical where two dev servers (React on 5173, Express on 5000) must run simultaneously demonstrate running both in separate terminals before students attempt it.</li> <li>• CORS errors are the single most common blocker in this session show what a CORS error looks like in the browser console before students hit it themselves.</li> <li>• Stress that frontend state should be updated based on the backend response, not assumed this is the core integration concept being tested.</li> <li>• Reserve the last 15 minutes for a full end-to-end demo check: create, edit, delete, and refresh.</li> </ul>                                                                                                                                                                                                                                                                                 |                                                                       |
|                                                | <b>Common Mistakes</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                       |
| <b>Common Mistakes</b>                         | <ul style="list-style-type: none"> <li>• Forgetting to start the backend server, then debugging the frontend for an unrelated 'fetch failed' error.</li> <li>• Not enabling CORS, resulting in browser console errors blocking all requests.</li> <li>• Updating local UI state optimistically without confirming the backend write actually succeeded.</li> <li>• Hardcoding the backend URL inconsistently across multiple files instead of using one shared base URL constant.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                       |
|                                                | <b>Troubleshooting Guide</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                       |
| <b>Symptom</b>                                 | <b>Likely Cause</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | <b>Fix</b>                                                            |
| CORS error in browser console                  | cors middleware not enabled on Express server                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Run npm install cors and add app.use(cors()) before route definitions |

|                                       |                                                             |                                                                                                   |
|---------------------------------------|-------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| fetch failed / Network Error          | Backend server not running or wrong port                    | Confirm Express server is running on port 5000 in a separate terminal                             |
| UI shows stale data after edit/delete | Local state not updated after successful API call           | Re-fetch the task list or manually update state array after each write operation                  |
| Data disappears after refresh         | Still reading from local component state instead of backend | Confirm GET /tasks is called on mount and populates state from the database, not hardcoded values |

### Student Assignment / Post Laboratory Work

|                                     |                                                                                                                                                                                                                                                                                                                                                                                                 |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tasks</b>                        | <ul style="list-style-type: none"> <li>Review the Theory Concepts section and the assigned Coursera module before next week.</li> <li>Add a confirmation dialog before deleting a task.</li> <li>Add a simple toast notification for successful create/update/delete actions.</li> </ul>                                                                                                        |
| <b>Expected Repository Contents</b> | <p><b>GitHub Deliverables</b></p> <ul style="list-style-type: none"> <li>Two repositories (or one monorepo) clearly separating frontend and backend</li> <li>Working end-to-end flow: create, read, update, delete tasks from the UI</li> <li>README with instructions to run both frontend and backend locally</li> <li>At least one commit specifically for full-stack integration</li> </ul> |

### Rubrics (Practical Evaluation / Viva)

| Criteria (Marks)                           | Description                                                                                   |
|--------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>API Connection from React (5 marks)</b> | React correctly calls backend endpoints; base URL configured; no CORS errors in console.      |
| <b>Create Operation (4 marks)</b>          | New task submitted from React form; persisted in MongoDB; appears in UI without page refresh. |
| <b>Read Operation (3 marks)</b>            | Task list fetched from backend on load; renders correctly in React.                           |
| <b>Update Operation (4 marks)</b>          | Edit functionality updates task via API; MongoDB record changes reflected in UI.              |
| <b>Delete Operation (4 marks)</b>          | Delete button removes task via API; UI updates immediately without full reload.               |
| <b>Total: 20 marks</b>                     | <b>Passing Threshold: 14 / 20 (70%)</b>                                                       |

| Practical 7: Authentication and Middleware Pipeline |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CO/PO                                               | CO2, CO6 / PO3, PO5                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Objective                                           | To implement JWT-based authentication and input validation as part of the Express middleware pipeline.                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Prerequisites                                       | <ul style="list-style-type: none"> <li>• Practical 6 completed working full-stack Task Management application</li> <li>• Basic understanding of hashing and tokens (conceptual, covered in self-study material)</li> </ul>                                                                                                                                                                                                                                                                                                                                      |
| Coursera Pointer                                    | <b>Theory Concepts (Self-Study Reference)</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|                                                     | See Week 7 IBM: Node.js & MongoDB Developing Back-end Database Applications, Module 4 (authentication and authorization, JWT token generation, error handling middleware, request validation and sanitization).                                                                                                                                                                                                                                                                                                                                                 |
|                                                     | <b>Architecture / Diagram</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|                                                     | <p>POST /register → hash password → save User → 201<br/>         POST /login → verify password → sign JWT → return token</p> <p>Protected Route Flow:<br/>         Client Request (Authorization: Bearer &lt;token&gt;)<br/>             ↓<br/>             v<br/>             [Auth Middleware] → verifies token → req.user set<br/>             ↓<br/>             v<br/>             [Validation Middleware] → checks input<br/>             ↓<br/>             v<br/>             Controller (task routes)</p>                                              |
| <b>Problem Definition</b>                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Problem Statement                                   | <ul style="list-style-type: none"> <li>• Add user registration and login functionality to the Task Management application.</li> <li>• Hash passwords using bcrypt before saving the User document to MongoDB.</li> <li>• Implement JWT token generation on successful login, with a reasonable expiry (e.g., 1 hour).</li> <li>• Protect all task routes using an authentication middleware that verifies the JWT.</li> <li>• Add server-side input validation that rejects malformed requests (e.g., missing title) before they reach the database.</li> </ul> |
| Key Questions / Analysis                            | <ul style="list-style-type: none"> <li>• Why must passwords be hashed before storage instead of saved as plain text, even in a lab/demo project?</li> <li>• What does the authentication middleware actually verify, and what happens if the token is missing or expired?</li> <li>• Why should input validation happen on the server even if the frontend already validates the same fields?</li> </ul>                                                                                                                                                        |
| Supplementary Problems                              | <ul style="list-style-type: none"> <li>• Add a /me endpoint that returns the currently logged-in user's details using the decoded JWT.</li> <li>• Implement token expiry handling on the frontend redirect to login if a protected request returns 401.</li> <li>• Add a logout mechanism that clears the token from the client.</li> </ul>                                                                                                                                                                                                                     |
| Key Skills Addressed                                | <ul style="list-style-type: none"> <li>• Hashing passwords securely using bcrypt</li> <li>• Generating and verifying JWT tokens</li> <li>• Writing authentication middleware to protect routes</li> <li>• Implementing server-side input validation and sanitization</li> </ul>                                                                                                                                                                                                                                                                                 |

|                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                       |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Learning Outcome                        | Students will be able to implement a complete authentication flow using JWT, protect backend routes with middleware, and enforce server-side input validation.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                                                                                       |
| Lab Session Step-by-Step                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                       |
| Steps                                   | <div>1. Install required packages:<br/>npm install bcryptjs jsonwebtoken</div> <div>2. Create a User schema with email and hashed password fields.</div> <div>3. Implement the register route hash the password before saving:<br/>const hashedPassword = await bcrypt.hash(password, 10);<br/>const user = await User.create({ email, password: hashedPassword });</div> <div>4. Implement the login route compare password and sign a JWT:<br/>const isMatch = await bcrypt.compare(password, user.password);<br/>const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: '1h' });</div> <div>5. Create an auth middleware that verifies the token from the Authorization header:<br/>const token = req.headers.authorization?.split(' ')[1];<br/>const decoded = jwt.verify(token, process.env.JWT_SECRET);<br/>req.user = decoded;</div> <div>6. Apply the auth middleware to all task routes.</div> <div>7. Add a validation middleware that checks required fields exist before reaching the controller.</div> <div>8. Test the full flow in Postman: register → login → copy token → access protected task routes.</div> |                                                                                       |
| Tools / Technology / Resources Required | <ul style="list-style-type: none"><li>• Node.js (v18+), npm</li><li>• bcryptjs, jsonwebtoken</li><li>• Express.js, Mongoose</li><li>• Postman (for token-based testing)</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                       |
| Total Hours                             | <ul style="list-style-type: none"><li>• Lab Session: 2 hours</li><li>• Implementation: 1.5 hours</li><li>• Testing / Modification: 0.5 hours</li><li>• Total Engagement: 2 hours</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                       |
| Faculty Teaching Guide                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                       |
| Guidance Notes                          | <ul style="list-style-type: none"><li>• Walk through why bcrypt and not plain SHA hashing is used bcrypt is intentionally slow and salts automatically, which matters for security.</li><li>• Demonstrate testing protected routes in Postman using the Authorization header with Bearer &lt;token&gt; before students attempt the frontend.</li><li>• Emphasize JWT_SECRET must come from .env, never hardcoded connect this back to the secure storage principle in the syllabus.</li><li>• Reserve time to show what happens with an expired or tampered token this builds real understanding of why verification matters.</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                                                                                       |
| Common Mistakes                         | <ul style="list-style-type: none"><li>• Storing passwords as plain text or using a reversible encryption instead of bcrypt hashing.</li><li>• Forgetting to send the Authorization header when testing protected routes, then assuming the middleware is broken.</li><li>• Hardcoding the JWT secret directly in code instead of using an environment variable.</li><li>• Not handling the case where jwt.verify() throws on an invalid/expired token, crashing the server.</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                       |
| Troubleshooting Guide                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                       |
| Symptom                                 | Likely Cause                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Fix                                                                                   |
| 401 Unauthorized on every request       | Authorization header missing or malformed in Postman/client                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | Set header as Authorization: Bearer <token> exactly, including the space after Bearer |

|                                               |                                                                                      |                                                                                                     |
|-----------------------------------------------|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Server crashes on invalid token               | jwt.verify() not wrapped in try/catch                                                | Wrap token verification in try/catch and return 401 on failure instead of letting it throw uncaught |
| Login always fails even with correct password | Comparing plain password to hashed password directly instead of using bcrypt.compare | Use bcrypt.compare(plainPassword, hashedPassword) rather than ===                                   |
| JWT_SECRET is undefined                       | .env file not loaded or variable name mismatch                                       | Confirm dotenv.config() runs before JWT_SECRET is used and the .env key name matches exactly        |

### Student Assignment / Post Laboratory Work

#### Tasks

- Review the Theory Concepts section and the assigned Coursera module before next week.
- Add a /me endpoint that returns the logged-in user's details from the decoded token.
- Implement basic frontend handling for 401 responses (redirect to a login prompt).

#### GitHub Deliverables

#### Expected Repository Contents

- Same repository as Practical 6, updated with new commits
- Working register/login flow with JWT-protected task routes
- .env excluded via .gitignore; JWT\_SECRET never committed
- At least one commit specifically for authentication implementation

### Rubrics (Practical Evaluation / Viva)

| Criteria (Marks)                          | Description                                                                                                                           |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>User Registration (3 marks)</b>        | Registration endpoint accepts name, email, password; password hashed before saving to MongoDB.                                        |
| <b>JWT Login Flow (5 marks)</b>           | Login endpoint validates credentials; returns signed JWT token with expiry; tested via Postman.                                       |
| <b>Auth Middleware (5 marks)</b>          | Protected routes reject requests without valid token; return 401; valid token grants access.                                          |
| <b>Input Validation (4 marks)</b>         | At least 3 validation rules enforced (e.g., email format, password length, required fields); invalid input returns descriptive error. |
| <b>Error Response Structure (3 marks)</b> | All errors returned as consistent JSON with status code and message; no stack traces exposed.                                         |
| <b>Total: 20 marks</b>                    | <b>Passing Threshold: 14 / 20 (70%)</b>                                                                                               |

| Practical 8: Performance Optimization and Lazy Loading in React |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CO/PO                                                           | CO1 / PO3, PO5                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Objective                                                       | To improve frontend performance using lazy loading and code splitting techniques.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Prerequisites                                                   | <ul style="list-style-type: none"> <li>• Practicals 1–6 completed multi-route React app with full-stack integration</li> <li>• Basic understanding of how the browser loads and parses JavaScript bundles</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                              |
| Theory Concepts (Self-Study Reference)                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Coursera Pointer                                                | See Week 8 IBM: Developing Front-End Apps with React, Module 4 (advanced React features, hooks for optimization). Supplement with browser DevTools Performance/Network tab profiling.                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Architecture / Diagram                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Diagram                                                         | <p>Before (single bundle):</p> <pre>main.bundle.js  →  [Home][Projects][Contact] all loaded upfront</pre> <p>After (code-split):</p> <pre> main.bundle.js  → loads on first visit Home.chunk.js   → loaded only when /  is visited Projects.chunk.js → loaded only when /projects is visited Contact.chunk.js → loaded only when /contact is visited </pre>                                                                                                                                                                                                                                                                       |
| Problem Definition                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Problem Statement                                               | <ul style="list-style-type: none"> <li>• Optimize the React frontend of the Task Management application by implementing lazy loading for route-based components.</li> <li>• Apply <code>React.lazy()</code> and <code>Suspense</code> to at least the Projects and Contact routes (or equivalent route-based components).</li> <li>• Provide a meaningful fallback UI while a lazy-loaded chunk is loading.</li> <li>• Measure and compare bundle size and load time before and after optimization using browser developer tools.</li> <li>• Document the before/after comparison with screenshots or recorded values.</li> </ul> |
| Key Questions / Analysis                                        | <ul style="list-style-type: none"> <li>• What is the difference between the initial bundle and a lazy-loaded chunk in terms of when each is downloaded?</li> <li>• Why does lazy loading improve perceived performance even though the total amount of code downloaded eventually stays the same?</li> <li>• In what situations would lazy loading not be worth the added complexity (e.g., a very small app)?</li> </ul>                                                                                                                                                                                                         |
| Supplementary Problems                                          | <ul style="list-style-type: none"> <li>• Lazy load a heavy third-party component (e.g., a chart library) only when a specific page needs it.</li> <li>• Add a minimum-delay fallback (e.g., 300ms) to avoid a flickering loading state on fast connections.</li> <li>• Use the React DevTools Profiler to identify and document one unnecessary re-render in the app.</li> </ul>                                                                                                                                                                                                                                                  |
| Key Skills Addressed                                            | <ul style="list-style-type: none"> <li>• Implementing route-based code splitting using <code>React.lazy()</code> and <code>Suspense</code></li> <li>• Reading and interpreting the browser Network tab for chunk loading behavior</li> <li>• Measuring bundle size using build tooling output</li> <li>• Communicating a before/after performance comparison clearly</li> </ul>                                                                                                                                                                                                                                                   |
| Learning Outcome                                                | Students will be able to apply code splitting and lazy loading in React to reduce initial bundle size, and will be able to measure and explain the resulting performance impact.                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

| Lab Session Step-by-Step                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                             |
|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| Steps                                             | <div>1. Record baseline metrics before optimization build the app and check bundle size:<br/>npm run build<br/># note the output bundle size shown in the terminal</div> <div>2. Open the Network tab in DevTools, reload the app, and note the total JS transferred and load time.</div> <div>3. Convert route imports to lazy imports:<br/>import { lazy, Suspense } from 'react';<br/>const Projects = lazy(() =&gt; import('./pages/Projects'));<br/>const Contact = lazy(() =&gt; import('./pages/Contact'));</div> <div>4. Wrap the Routes block with Suspense and a fallback:<br/>&lt;Suspense fallback=&lt;div&gt;Loading page...&lt;/div&gt;&gt;<br/>  &lt;Routes&gt;<br/>    &lt;Route path="/projects" element={&lt;Projects /&gt;} /&gt;<br/>    &lt;Route path="/contact" element={&lt;Contact /&gt;} /&gt;<br/>  &lt;/Routes&gt;<br/>&lt;/Suspense&gt;</div> <div>5. Rebuild the app and confirm separate chunk files are generated for each lazy route.</div> <div>6. Reload with Network tab open, throttle to "Slow 3G", and observe the fallback UI rendering before the chunk loads.</div> <div>7. Record post-optimization metrics and prepare the before/after comparison.</div> |                                                             |
| Tools / Technology / Resources Required           | <ul style="list-style-type: none"><li>• Node.js (v18+), npm</li><li>• React 18+ (lazy, Suspense)</li><li>• Browser DevTools (Network + Performance tabs)</li><li>• Vite build output</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                                                             |
| Total Hours                                       | <ul style="list-style-type: none"><li>• Lab Session: 2 hours</li><li>• Implementation: 1.5 hours</li><li>• Testing / Modification: 0.5 hours</li><li>• Total Engagement: 2 hours</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                             |
| Faculty Teaching Guide                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                             |
| Guidance Notes                                    | <ul style="list-style-type: none"><li>• Tell students in advance to capture baseline screenshots before making any changes this is required evidence for the rubric and cannot be reconstructed afterward.</li><li>• Demonstrate Network throttling (Slow 3G) live so the fallback UI is actually visible on fast lab Wi-Fi the loading state may flash too quickly to notice otherwise.</li><li>• Clarify that lazy loading delays when code downloads, not how much code downloads in total over a full session.</li><li>• Walk through reading the Vite build output chunk list together once before students interpret their own.</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                             |
| Common Mistakes                                   | <ul style="list-style-type: none"><li>• Not capturing baseline (before) metrics, making the comparison impossible to demonstrate.</li><li>• Wrapping individual components in Suspense unnecessarily instead of wrapping at the route level.</li><li>• Forgetting that React.lazy() only works with default exports, causing import errors.</li><li>• Testing only on fast lab Wi-Fi without throttling, so the fallback UI is never actually observed.</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                                             |
| Troubleshooting Guide                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                             |
| Symptom                                           | Likely Cause                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Fix                                                         |
| Element type is invalid error after adding lazy() | Lazy-loaded component is not a default export                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | Ensure the component file uses export default ComponentName |

|                                                                |                                                          |                                                                                                   |
|----------------------------------------------------------------|----------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Fallback UI never appears                                      | Chunk loads too fast on local network to observe         | Enable Network throttling (Slow 3G) in DevTools to simulate a slower connection                   |
| No separate chunk files after build                            | Static import still used instead of dynamic import()     | Confirm the route component uses lazy(() => import('./Component')) syntax, not a top-level import |
| Suspense fallback shows for the entire app, not just one route | Suspense wraps too broad a section of the component tree | Move Suspense to wrap only the Routes block, not the entire App component                         |

### Student Assignment / Post Laboratory Work

#### Tasks

- Review the Theory Concepts section and the assigned Coursera module before next week.
- Lazy load one additional heavy component (e.g., a chart or large list) beyond the required routes.
- Use React DevTools Profiler to identify and note one unnecessary re-render in the app.

#### GitHub Deliverables

#### Expected Repository Contents

- Same repository as Practical 6/7, updated with new commits
- Lazy loading applied to at least 2 route-based components with Suspense fallback
- Before/after screenshots or recorded metrics included in the README or a docs folder
- At least one commit specifically for performance optimization

### Rubrics (Practical Evaluation / Viva)

| Criteria (Marks)                             | Description                                                                                               |
|----------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>Lazy Loading Implementation (6 marks)</b> | React.lazy() applied to at least 2 route-based components; Suspense wrapper with fallback UI present.     |
| <b>Code Splitting Evidence (5 marks)</b>     | Browser Network tab shows separate chunks loading per route; demonstrated live during evaluation.         |
| <b>Before/After Comparison (5 marks)</b>     | Screenshot or recording of bundle size and load time before and after optimization; difference explained. |
| <b>Fallback UI (4 marks)</b>                 | Suspense fallback renders correctly during lazy load; does not crash if chunk is slow.                    |
| <b>Total: 20 marks</b>                       | <b>Passing Threshold: 14 / 20 (70%)</b>                                                                   |

| Practical 9: In-Memory Caching and Query Optimization |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CO/PO                                                 | CO2, CO4 / PO3, PO5                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Objective                                             | To implement server-side caching and measure its impact on API response time.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Prerequisites                                         | <ul style="list-style-type: none"> <li>• Practicals 4–7 completed working Node/Express/MongoDB backend with authentication</li> <li>• Basic understanding of why repeated database reads are expensive at scale</li> </ul>                                                                                                                                                                                                                                                                                                                                         |
| Theory Concepts (Self-Study Reference)                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Coursera Pointer                                      | See Week 9 IBM: Node.js & MongoDB Developing Back-end Database Applications, Module 5 (optimization techniques, back-end performance, scaling approaches, load balancing overview).                                                                                                                                                                                                                                                                                                                                                                                |
| Diagram                                               | Architecture / Diagram                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|                                                       | <pre> GET /tasks request       v Cache check (node-cache)       +-- HIT → return cached data immediately       +-- MISS → query MongoDB → store in cache → return data  POST/PUT/DELETE /tasks       v Write to MongoDB → invalidate cache key </pre>                                                                                                                                                                                                                                                                                                              |
| Problem Statement                                     | Problem Definition                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|                                                       | <ul style="list-style-type: none"> <li>• Add in-memory caching to the Task Management backend using node-cache.</li> <li>• Cache the response of GET /tasks (all tasks) with a reasonable TTL (e.g., 60 seconds).</li> <li>• Invalidate the cache on any write operation POST, PUT, or DELETE so stale data is never served.</li> <li>• Record and compare API response times with and without caching using Postman or Thunder Client.</li> <li>• Document the measured difference with at least 3 sample readings for each case (cached vs uncached).</li> </ul> |
| Key Questions / Analysis                              | <ul style="list-style-type: none"> <li>• Why must the cache be invalidated on every write operation, and what would happen to data correctness if it were not?</li> <li>• What is a reasonable TTL (time-to-live) for cached data in a task management context, and what trade-off does TTL length represent?</li> <li>• Why is in-memory caching (node-cache) not suitable for a multi-server/multi-instance deployment, even though it works fine in this lab?</li> </ul>                                                                                        |
| Supplementary Problems                                | <ul style="list-style-type: none"> <li>• Cache the GET /tasks/:id single-task endpoint separately from the all-tasks endpoint.</li> <li>• Add a cache-hit/cache-miss counter and expose it via a debug endpoint.</li> <li>• Experiment with different TTL values and document how it affects perceived staleness versus performance.</li> </ul>                                                                                                                                                                                                                    |
| Key Skills Addressed                                  | <ul style="list-style-type: none"> <li>• Implementing in-memory caching using node-cache</li> <li>• Designing correct cache invalidation logic for write operations</li> <li>• Measuring and comparing API response times empirically</li> <li>• Reasoning about the trade-offs of caching strategies</li> </ul>                                                                                                                                                                                                                                                   |
| Learning Outcome                                      | Students will be able to implement in-memory caching for a REST API, ensure cache correctness through invalidation, and measure the performance impact using real response time data.                                                                                                                                                                                                                                                                                                                                                                              |

| Lab Session Step-by-Step                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                       |
|------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Steps                                                | <div>1. Install node-cache:<br/>npm install node-cache</div> <div>2. Initialize a cache instance in a shared module:<br/>const NodeCache = require('node-cache');<br/>const cache = new NodeCache({ stdTTL: 60 });<br/>module.exports = cache;</div> <div>3. Update the GET /tasks route to check the cache first:<br/>const cached = cache.get('all_tasks');<br/>if (cached) return res.json(cached);<br/>const tasks = await Task.find();<br/>cache.set('all_tasks', tasks);<br/>res.json(tasks);</div> <div>4. Invalidate the cache inside POST, PUT, and DELETE handlers after a successful write:<br/>cache.del('all_tasks');</div> <div>5. In Postman, send GET /tasks 3 times and record response time for each (look at the "Time" field).</div> <div>6. Temporarily disable caching (comment out the cache check) and repeat the same 3 requests for comparison.</div> <div>7. Record both sets of readings in a simple table for the lab journal.</div> |                                                                                       |
| Tools / Technology / Resources Required              | <ul style="list-style-type: none"><li>• Node.js (v18+), npm</li><li>• node-cache</li><li>• Express.js, Mongoose</li><li>• Postman or Thunder Client (for response time measurement)</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                       |
| Total Hours                                          | <ul style="list-style-type: none"><li>• Lab Session: 2 hours</li><li>• Implementation: 1.5 hours</li><li>• Testing / Modification: 0.5 hours</li><li>• Total Engagement: 2 hours</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                                                                                       |
| Faculty Teaching Guide                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                       |
| Guidance Notes                                       | <ul style="list-style-type: none"><li>• Tell students explicitly to record uncached response times BEFORE adding the cache check, or the comparison cannot be done fairly afterward.</li><li>• Clarify that node-cache is process-local memory explain briefly why this would not work correctly if the app ran across multiple server instances (sets up the later Unit 4 scaling discussion).</li><li>• Walk through a live example of forgetting cache invalidation, showing students a stale response after an update, to make the risk concrete.</li><li>• Encourage at least 3 repeated readings per condition rather than a single measurement, since response time can vary.</li></ul>                                                                                                                                                                                                                                                                    |                                                                                       |
| Common Mistakes                                      | <ul style="list-style-type: none"><li>• Forgetting to invalidate the cache after a write, leading to stale data being served indefinitely within the TTL window.</li><li>• Caching per-request without a stable cache key, causing every request to be treated as a cache miss.</li><li>• Only taking a single response time reading instead of multiple samples, making the comparison unreliable.</li><li>• Setting a TTL so short that the cache effectively never gets hit during testing.</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                       |
| Troubleshooting Guide                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                       |
| Symptom                                              | Likely Cause                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | Fix                                                                                   |
| Updated task not reflected in GET response           | Cache not invalidated after PUT/DELETE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | Add cache.del('all_tasks') inside every write route handler                           |
| No measurable difference between cached and uncached | TTL too short or cache key changing on every request                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | Use a fixed, consistent cache key string and a TTL of at least 60 seconds for testing |

|                                         |                                                |                                                                                                              |
|-----------------------------------------|------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Cache works but server restarts wipe it | node-cache is in-memory and resets on restart  | This is expected behavior for in-memory caching note it in the lab journal observation as a known limitation |
| Response time barely changes            | MongoDB query was already fast (small dataset) | Add more sample task documents to the database to make the cache benefit more visible                        |

### Student Assignment / Post Laboratory Work

|                                     |                                                                                                                                                                                                                                                                                                                                                                              |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tasks</b>                        | <ul style="list-style-type: none"> <li>• Review the Theory Concepts section and the assigned Coursera module before next week.</li> <li>• Cache the single-task GET /tasks/:id endpoint separately from the all-tasks endpoint.</li> <li>• Add a simple cache-hit/cache-miss counter exposed via a debug endpoint.</li> </ul>                                                |
|                                     | <b>GitHub Deliverables</b>                                                                                                                                                                                                                                                                                                                                                   |
| <b>Expected Repository Contents</b> | <ul style="list-style-type: none"> <li>• Same repository as Practical 4/5, updated with new commits</li> <li>• node-cache implementation with correct invalidation on all write operations</li> <li>• Response time comparison data (cached vs uncached) included in README or docs folder</li> <li>• At least one commit specifically for caching implementation</li> </ul> |

### Rubrics (Practical Evaluation / Viva)

| Criteria (Marks)                          | Description                                                                                                   |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>node-cache Setup (4 marks)</b>         | node-cache installed and initialized correctly; cache instance used in route file.                            |
| <b>Cache on GET (5 marks)</b>             | GET all tasks checks cache first; returns cached data if available; fetches from DB only on cache miss.       |
| <b>Cache Invalidation (5 marks)</b>       | Cache cleared correctly on POST, PUT, DELETE operations; stale data does not persist after write.             |
| <b>Response Time Comparison (6 marks)</b> | Response times recorded with and without caching using Postman; measurable difference demonstrated and noted. |
| <b>Total: 20 marks</b>                    | <b>Passing Threshold: 14 / 20 (70%)</b>                                                                       |

## Practical 10: Asynchronous Processing with Event-Driven Architecture

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CO/PO</b>                    | CO4 / PO3, PO5                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Objective</b>                | To implement asynchronous background processing using Node.js native EventEmitter without external dependencies.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Prerequisites</b>            | <ul style="list-style-type: none"> <li>• Practicals 4–9 completed working Node/Express/MongoDB backend with caching</li> <li>• Basic understanding of synchronous vs asynchronous execution in JavaScript</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|                                 | <b>Theory Concepts (Self-Study Reference)</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Coursera Pointer</b>         | No dedicated Coursera module for this topic refer to the Node.js official documentation on Events ( <a href="https://nodejs.org/en/docs/guides/event-loop-timers-and-nexttick">nodejs.org/en/docs/guides/event-loop-timers-and-nexttick</a> ), supplemented by the Node.js Crash Course on the Traversy Media YouTube channel.                                                                                                                                                                                                                                                                                                                                                               |
|                                 | <b>Architecture / Diagram</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Diagram</b>                  | <pre> POST /tasks request           v Save task to MongoDB           v emit('task-created', taskData) → API responds immediately (201)           v (handled asynchronously, after response sent) Notification Listener           └─ logs: task title, timestamp, assigned user         </pre>                                                                                                                                                                                                                                                                                                                                                                                                |
|                                 | <b>Problem Definition</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Problem Statement</b>        | <ul style="list-style-type: none"> <li>• Add a background notification system to the Task Management application using Node.js built-in EventEmitter.</li> <li>• When a new task is created via the API, emit a task-created event handled asynchronously.</li> <li>• The event handler must log a notification (task title, timestamp, assigned user) without blocking the main response.</li> <li>• The main API response must return immediately, before the event handler finishes executing.</li> <li>• Demonstrate the difference between synchronous and asynchronous handling by logging timestamps at both the API response point and the event handler execution point.</li> </ul> |
| <b>Key Questions / Analysis</b> | <ul style="list-style-type: none"> <li>• Why does emitting an event not block the API response, even though both run on the same Node.js process?</li> <li>• What would happen to API response time if the notification logic were placed directly inside the POST route instead of in an event handler?</li> <li>• Why is EventEmitter a reasonable choice for this scale of application, but not for a production system handling millions of events per day?</li> </ul>                                                                                                                                                                                                                   |
| <b>Supplementary Problems</b>   | <ul style="list-style-type: none"> <li>• Add a second event, task-deleted, with its own listener that logs a different message.</li> <li>• Add an error event listener that catches and logs any error thrown inside the task-created handler.</li> <li>• Simulate a slow notification handler (e.g., setTimeout of 2 seconds) and confirm the API still responds instantly.</li> </ul>                                                                                                                                                                                                                                                                                                      |
| <b>Key Skills Addressed</b>     | <ul style="list-style-type: none"> <li>• Using Node.js built-in EventEmitter to decouple side effects from the main request flow</li> <li>• Designing non-blocking background processing without external message queue infrastructure</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                            |

|                                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                | <ul style="list-style-type: none"> <li>• Understanding the Node.js event loop at a practical, applied level</li> <li>• Comparing synchronous and asynchronous execution using timestamp evidence</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Learning Outcome</b>                        | Students will be able to implement non-blocking, event-driven background processing using Node's built-in EventEmitter and explain how it keeps the main request-response cycle fast.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Lab Session Step-by-Step</b>                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Steps</b>                                   | <ol style="list-style-type: none"> <li>1. Create a dedicated events.js module:<br/> <pre>const EventEmitter = require('events'); class TaskEvents extends EventEmitter {} module.exports = new TaskEvents();</pre> </li> <li>2. Register a listener for the task-created event, ideally in a separate listeners.js file:<br/> <pre>const taskEvents = require('./events'); taskEvents.on('task-created', (task) =&gt; {   console.log(`[Notification] Task "\${task.title}" created at \${new Date().toISOString()}`); });</pre> </li> <li>3. Inside the POST /tasks route, emit the event right after saving the task, then respond immediately:<br/> <pre>const task = await Task.create(req.body); console.log(`[API] Response sent at \${new Date().toISOString()}`); res.status(201).json(task); taskEvents.emit('task-created', task);</pre> </li> <li>4. Run the server and create a task via Postman; observe the console log order.</li> <li>5. Add an artificial delay inside the listener (e.g., a loop or setTimeout) to make the asynchronous behavior more visible.</li> <li>6. Confirm and record that the API response timestamp appears before the listener's completion timestamp in the console.</li> </ol> |
| <b>Tools / Technology / Resources Required</b> | <ul style="list-style-type: none"> <li>• Node.js (v18+) built-in events module</li> <li>• Express.js, Mongoose</li> <li>• Postman or Thunder Client</li> <li>• Visual Studio Code</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Total Hours</b>                             | <ul style="list-style-type: none"> <li>• Lab Session: 2 hours</li> <li>• Implementation: 1.5 hours</li> <li>• Testing / Modification: 0.5 hours</li> <li>• Total Engagement: 2 hours</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Faculty Teaching Guide</b>                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Guidance Notes</b>                          | <ul style="list-style-type: none"> <li>• Clarify upfront that this practical replaces a message-queue-based approach (e.g., RabbitMQ/Bull/Redis) which is out of scope for this course. EventEmitter is the deliberate, simpler substitute.</li> <li>• Emphasize that emit() does not wait for listeners to finish; this is the core concept being tested and is easy to misunderstand as 'asynchronous magic'.</li> <li>• Use the console.log timestamp ordering as the main proof point during evaluation rather than relying on student explanation alone.</li> <li>• Connect this practical back to Practical 9's caching discussion; both are examples of decoupling work from the critical request path.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Common Mistakes</b>                         | <ul style="list-style-type: none"> <li>• Placing taskEvents.emit() before res.status(201).json(task), which still works but obscures the ordering being demonstrated; emit after the response logic is clearer pedagogically.</li> <li>• Writing notification logic directly inside the route handler instead of in a separate listener, defeating the purpose of the exercise.</li> <li>• Not adding any artificial delay or distinguishing log statements, making the synchronous vs asynchronous difference invisible in the console output.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |

|                                            | <ul style="list-style-type: none"><li>• Forgetting to register the listener (<code>taskEvents.on(...)</code>) before the event is ever emitted, causing the event to silently do nothing.</li></ul>                                                                                                                                                                                             |                                                                                                                       |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Troubleshooting Guide                      |                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                                       |
| Symptom                                    | Likely Cause                                                                                                                                                                                                                                                                                                                                                                                    | Fix                                                                                                                   |
| Notification log never appears             | Listener file never required/imported, so <code>.on()</code> was never registered                                                                                                                                                                                                                                                                                                               | Ensure the listeners file is required once at server startup, e.g., in <code>server.js</code>                         |
| API response feels delayed                 | Listener logic accidentally placed before the response is sent, or <code>emit()</code> called synchronously before <code>res.json()</code>                                                                                                                                                                                                                                                      | Move <code>res.status(201).json(task)</code> to execute before <code>taskEvents.emit(...)</code>                      |
| Unhandled 'error' event crashes the server | EventEmitter has a special case: an unhandled 'error' event throws                                                                                                                                                                                                                                                                                                                              | Always register an error listener if emitting custom error events, or avoid using the reserved 'error' event name     |
| Timestamps appear identical                | No artificial delay added to the listener to make async timing visible                                                                                                                                                                                                                                                                                                                          | Add a short <code>setTimeout</code> or loop inside the listener to exaggerate the timing difference for demonstration |
| Student Assignment / Post Laboratory Work  |                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                                       |
| Tasks                                      | <ul style="list-style-type: none"><li>• Review the Theory Concepts section and the Node.js Events documentation before next week.</li><li>• Add a task-deleted event with its own listener and distinct log message.</li><li>• Add an error event listener that catches and logs any error thrown inside the task-created handler.</li></ul>                                                    |                                                                                                                       |
| GitHub Deliverables                        |                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                                       |
| Expected Repository Contents               | <ul style="list-style-type: none"><li>• Same repository as Practical 4/5/9, updated with new commits</li><li>• Working EventEmitter-based notification system with at least one custom event</li><li>• Console log evidence (screenshot or recording) showing response-before-handler timestamp ordering</li><li>• At least one commit specifically for event-driven async processing</li></ul> |                                                                                                                       |
| Rubrics (Practical Evaluation / Viva)      |                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                                       |
| Criteria (Marks)                           | Description                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                                                       |
| EventEmitter Setup (4 marks)               | Node.js built-in EventEmitter imported and instantiated correctly in a dedicated module.                                                                                                                                                                                                                                                                                                        |                                                                                                                       |
| Event Emission on Task Create (5 marks)    | task-created event emitted inside POST endpoint; event carries task title, timestamp, and relevant data.                                                                                                                                                                                                                                                                                        |                                                                                                                       |
| Async Handler (5 marks)                    | Event listener processes notification asynchronously; main API response returns before handler completes.                                                                                                                                                                                                                                                                                       |                                                                                                                       |
| Timestamp Logging Evidence (6 marks)       | Console shows API response timestamp before notification handler timestamp; difference demonstrated live.                                                                                                                                                                                                                                                                                       |                                                                                                                       |
| Total: 20 marks                            | Passing Threshold: 14 / 20 (70%)                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                       |

| Practical 11: Containerization with Docker and Docker Compose |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CO/PO                                                         | CO5 / PO3, PO5                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Objective                                                     | To containerize the full-stack application using Docker and orchestrate services using Docker Compose.                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Prerequisites                                                 | <ul style="list-style-type: none"> <li>• Practicals 1–10 completed working full-stack Task Management application</li> <li>• Docker Desktop (or Docker Engine) installed and verified on the lab machine</li> </ul>                                                                                                                                                                                                                                                                                                                           |
|                                                               | <b>Theory Concepts (Self-Study Reference)</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Coursera Pointer                                              | See Week 11 IBM: Introduction to Containers with Docker, Kubernetes & OpenShift (focus only on the Docker fundamentals and Docker Compose sections; Kubernetes/OpenShift content is out of scope for this course).                                                                                                                                                                                                                                                                                                                            |
|                                                               | <b>Architecture / Diagram</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Diagram                                                       | <pre> docker-compose.yml ├── frontend  (React, built + served, port 5173) ├── backend   (Node/Express, port 5000) └── mongodb   (official mongo image, port 27017)  Network: app-network (bridge) frontend → backend → mongodb All 3 services start together via: docker-compose up </pre>                                                                                                                                                                                                                                                    |
|                                                               | <b>Problem Definition</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Problem Statement                                             | <ul style="list-style-type: none"> <li>• Write a Dockerfile for the React frontend of the Task Management application.</li> <li>• Write a Dockerfile for the Node/Express backend.</li> <li>• Create a docker-compose.yml file that orchestrates all 3 services: frontend, backend, and MongoDB.</li> <li>• Configure correct port mappings and an internal Docker network so the services can communicate.</li> <li>• The entire application must start with a single docker-compose up command, with no manual steps in between.</li> </ul> |
| Key Questions / Analysis                                      | <ul style="list-style-type: none"> <li>• Why does the backend container need to reference the MongoDB container by its service name (e.g., mongodb) rather than localhost?</li> <li>• What is the difference between EXPOSE in a Dockerfile and the ports mapping in docker-compose.yml?</li> <li>• Why is it considered bad practice to copy node_modules into a Docker image instead of running npm install inside the container?</li> </ul>                                                                                                |
| Supplementary Problems                                        | <ul style="list-style-type: none"> <li>• Add a .dockerignore file to exclude node_modules and .env from the build context.</li> <li>• Use a multi-stage build for the React frontend to reduce final image size.</li> <li>• Add a named volume for MongoDB so data persists across container restarts.</li> </ul>                                                                                                                                                                                                                             |
| Key Skills Addressed                                          | <ul style="list-style-type: none"> <li>• Writing Dockerfiles for both a frontend and backend Node-based application</li> <li>• Orchestrating multi-container applications using Docker Compose</li> <li>• Configuring inter-container networking and port mapping</li> <li>• Running and verifying a complete application stack with a single command</li> </ul>                                                                                                                                                                              |
| Learning Outcome                                              | Students will be able to containerize a full-stack application using Docker and orchestrate multiple services together using Docker Compose, running the entire stack with one command.                                                                                                                                                                                                                                                                                                                                                       |

| Lab Session Step-by-Step                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Steps                                   | <ol style="list-style-type: none"> <li>1. Verify Docker is installed and running:<br/> <pre>docker --version docker compose version</pre> </li> <li>2. Create a Dockerfile in the backend folder:<br/> <pre>FROM node:18 WORKDIR /app COPY package*.json ./ RUN npm install COPY . . EXPOSE 5000 CMD ["node", "server.js"]</pre> </li> <li>3. Create a Dockerfile in the frontend folder:<br/> <pre>FROM node:18 AS build WORKDIR /app COPY package*.json ./ RUN npm install COPY . . RUN npm run build EXPOSE 5173 CMD ["npm", "run", "preview", "--", "--host"]</pre> </li> <li>4. Create docker-compose.yml at the project root defining all 3 services with build context, ports, and depends_on.</li> <li>5. Update the backend MongoDB connection string to use the service name (e.g., mongodb://mongodb:27017/taskdb) instead of localhost.</li> <li>6. Run the entire stack:<br/> <pre>docker-compose up --build</pre> </li> <li>7. Confirm all 3 containers are running and the app is accessible end-to-end:<br/> <pre>docker ps</pre> </li> </ol> |
| Tools / Technology / Resources Required | <ul style="list-style-type: none"> <li>• Docker Desktop / Docker Engine</li> <li>• Docker Compose</li> <li>• Node.js (for local reference, not required at runtime once containerized)</li> <li>• Terminal / Command line</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Total Hours                             | <ul style="list-style-type: none"> <li>• Lab Session: 2 hours</li> <li>• Implementation: 1.5 hours</li> <li>• Testing / Modification: 0.5 hours</li> <li>• Total Engagement: 2 hours</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Faculty Teaching Guide                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Guidance Notes                          | <ul style="list-style-type: none"> <li>• This is an awareness-level practical, not a production Docker deep dive the goal is for students to see and run a working multi-container setup, not master Docker internals.</li> <li>• Pre-pull the node:18 and mongo images on lab machines before the session if internet bandwidth is limited, to avoid losing lab time to image downloads.</li> <li>• Demonstrate the localhost vs service-name distinction live this is the single most common point of confusion when containerizing a previously local app.</li> <li>• Reserve the last 15 minutes specifically for the single-command verification (docker-compose up) as a class checkpoint.</li> </ul>                                                                                                                                                                                                                                                                                                                                                   |
| Common Mistakes                         | <ul style="list-style-type: none"> <li>• Using localhost or 127.0.0.1 in the backend's MongoDB connection string instead of the Compose service name.</li> <li>• Forgetting to expose/map the correct ports, making the app inaccessible from the host browser.</li> <li>• Not excluding node_modules via .dockerignore, resulting in extremely slow and bloated image builds.</li> <li>• Running docker-compose up without --build after changing a Dockerfile, so old cached layers are used.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Troubleshooting Guide                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |

| Symptom                                     | Likely Cause                                                         | Fix                                                                                                         |
|---------------------------------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Backend cannot connect to MongoDB           | Connection string uses localhost instead of the Compose service name | Update MONGO_URI to use the service name defined in docker-compose.yml, e.g. mongodb://mongodb:27017/taskdb |
| Port already in use error                   | A previous container or local process is already bound to that port  | Run docker ps to find and stop the conflicting container, or change the host-side port mapping              |
| Build is extremely slow                     | node_modules being copied into the build context                     | Add a .dockerignore file with node_modules listed                                                           |
| Changes to code not reflected after rebuild | Old image layers cached                                              | Run docker-compose up --build to force a fresh image build                                                  |

### Student Assignment / Post Laboratory Work

#### Tasks

- Review the Theory Concepts section and the assigned Coursera module before next week.
- Add a .dockerignore file excluding node\_modules and .env.
- Add a named volume for MongoDB so task data persists across container restarts.

#### GitHub Deliverables

#### Expected Repository Contents

- Same repository as previous practicals, updated with new commits
- Dockerfiles for both frontend and backend, plus a working docker-compose.yml
- README updated with instructions to run the full stack via docker-compose up
- At least one commit specifically for containerization

### Rubrics (Practical Evaluation / Viva)

| Criteria (Marks)                     | Description                                                                                                             |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Dockerfile Frontend (4 marks)</b> | Valid Dockerfile for React app; correct base image; build and serve steps present.                                      |
| <b>Dockerfile Backend (4 marks)</b>  | Valid Dockerfile for Node/Express; correct base image; dependencies installed; app starts on container run.             |
| <b>Docker Compose File (6 marks)</b> | Compose file defines all 3 services (frontend, backend, MongoDB); correct port mappings; services linked via network.   |
| <b>Single Command Run (6 marks)</b>  | Entire app starts with docker-compose up without manual intervention; all 3 containers running confirmed via docker ps. |
| <b>Total: 20 marks</b>               | <b>Passing Threshold: 14 / 20 (70%)</b>                                                                                 |

| Practical 12: CI/CD Pipeline with GitHub Actions |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|--------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CO/PO                                            | CO5 / PO3, PO5                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Objective                                        | To automate testing and deployment of the application using a CI/CD pipeline.                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Prerequisites                                    | <ul style="list-style-type: none"> <li>• Practical 11 completed containerized full-stack application</li> <li>• GitHub account with the project repository already pushed</li> <li>• Basic understanding of YAML syntax</li> </ul>                                                                                                                                                                                                                                                                                                      |
| Coursera Pointer                                 | <b>Theory Concepts (Self-Study Reference)</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|                                                  | See Week 12 GitHub Actions Masterclass (Packt), Module 1 (workflow basics, triggers, jobs, steps) and Module 2 (CI/CD pipeline setup, automated testing on push).                                                                                                                                                                                                                                                                                                                                                                       |
| Architecture / Diagram                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Diagram                                          | <pre> git push origin main           v GitHub Actions Trigger (on: push)           v Job: build-and-test ├── Checkout code ├── Setup Node.js ├── npm install ├── Run tests ├── Report pass / fail status           v Green checkmark (pass) or Red X (fail) on GitHub </pre>                                                                                                                                                                                                                                                            |
| Problem Definition                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Problem Statement                                | <ul style="list-style-type: none"> <li>• Set up a GitHub Actions workflow for the Task Management application that runs automatically on every push to the main branch.</li> <li>• The pipeline must install dependencies (npm install) as a defined step.</li> <li>• The pipeline must run at least one automated test and report a clear pass/fail status.</li> <li>• Demonstrate a failed pipeline caused by a deliberately broken test.</li> <li>• Demonstrate a fixed pipeline that passes after the test is corrected.</li> </ul> |
| Key Questions / Analysis                         | <ul style="list-style-type: none"> <li>• What is the difference between Continuous Integration and Continuous Deployment, and which one does this practical actually demonstrate?</li> <li>• Why does running tests automatically on every push catch problems earlier than testing manually before a release?</li> <li>• What would happen to the pipeline if the test file path were incorrect would the workflow fail at the same step as a genuine test failure?</li> </ul>                                                         |
| Supplementary Problems                           | <ul style="list-style-type: none"> <li>• Add a second job that runs a linter (e.g., ESLint) before the test job.</li> <li>• Configure the workflow to run only on pull requests targeting main, not on every direct push.</li> <li>• Add a status badge to the README that reflects the current pipeline status.</li> </ul>                                                                                                                                                                                                             |
| Key Skills Addressed                             | <ul style="list-style-type: none"> <li>• Writing a GitHub Actions workflow YAML file</li> <li>• Configuring triggers, jobs, and steps for a CI pipeline</li> <li>• Writing at least one automated test that can run in CI</li> <li>• Reading and interpreting CI pipeline logs and pass/fail status</li> </ul>                                                                                                                                                                                                                          |
| Learning Outcome                                 | Students will be able to set up a basic CI pipeline using GitHub Actions that automatically installs dependencies and runs tests on every push, and will be able to diagnose a failing pipeline from its logs.                                                                                                                                                                                                                                                                                                                          |

## Lab Session Step-by-Step

### Steps

1. Install a basic testing library in the backend project:  
`npm install --save-dev jest supertest`
2. Write at least one simple test (e.g., testing the GET /tasks endpoint returns status 200):  

```
test('GET /tasks returns 200', async () => {  
  const res = await request(app).get('/tasks');  
  expect(res.statusCode).toBe(200);  
});
```
3. Create the workflow file at .github/workflows/ci.yml:  

```
name: CI Pipeline  
on:  
  push:  
    branches: [main]  
jobs:  
  build-and-test:  
    runs-on: ubuntu-latest  
    steps:  
      - uses: actions/checkout@v4  
      - uses: actions/setup-node@v4  
        with:  
          node-version: 18  
      - run: npm install  
      - run: npm test
```
4. Commit and push the workflow file; confirm it appears under the Actions tab on GitHub.
5. Deliberately break the test (change an expected value) and push again observe the red X / failed status.
6. Fix the test and push again observe the pipeline turn green.
7. Take screenshots of both the failed and passed pipeline runs for the lab journal.

### Tools / Technology / Resources Required

- GitHub account and repository
- GitHub Actions (no separate install required)
- Jest or a similar Node.js testing library
- Git CLI, Visual Studio Code

### Total Hours

- Lab Session: 2 hours
- Implementation: 1.5 hours
- Testing / Modification: 0.5 hours
- Total Engagement: 2 hours

## Faculty Teaching Guide

### Guidance Notes

- Confirm GitHub access and internet connectivity in the lab before this session the entire practical depends on it.
- Walk through the Actions tab UI together first so students know where to find pipeline runs and logs before debugging on their own.
- Make the deliberate test failure a required step, not optional seeing a red X and reading the failure log is the core learning moment.
- Keep the test itself trivial (e.g., a single status code check) the goal is understanding the pipeline mechanism, not testing depth.

### Common Mistakes

- Placing the workflow YAML file in the wrong path (must be exactly .github/workflows/<name>.yml).
- Incorrect YAML indentation, causing the workflow to fail to even register on GitHub.
- Writing a test that always passes trivially (e.g., expect(true).toBe(true)), which defeats the purpose of the demonstration.
- Forgetting to push the workflow file itself, then wondering why nothing appears under the Actions tab.

## Troubleshooting Guide

### Symptom

### Likely Cause

### Fix

|                                                     |                                                                                 |                                                                                      |
|-----------------------------------------------------|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Workflow does not appear under Actions tab          | File not located at .github/workflows/ or has incorrect extension               | Confirm the exact path .github/workflows/ci.yml and that it was committed and pushed |
| Workflow fails immediately with a YAML syntax error | Incorrect indentation in the YAML file                                          | YAML is indentation-sensitive; use consistent 2-space indentation and avoid tabs     |
| npm test fails with 'no test specified'             | package.json test script not configured to run Jest                             | Set "scripts": { "test": "jest" } in package.json                                    |
| Pipeline passes even with a broken test             | Test was written incorrectly and does not actually assert the intended behavior | Review the test logic to confirm it fails when the code is genuinely broken          |

### Student Assignment / Post Laboratory Work

#### Tasks

- Review the Theory Concepts section and the assigned Coursera module before next week.
- Add a second automated test covering a different endpoint.
- Add a status badge to the project README reflecting the current CI pipeline status.

#### GitHub Deliverables

#### Expected Repository Contents

- Same repository as previous practicals, updated with new commits
- Working .github/workflows/ci.yml with install and test steps
- Screenshots of both a failed and a passed pipeline run included in README or docs folder
- At least one commit specifically for CI pipeline setup

### Rubrics (Practical Evaluation / Viva)

| Criteria (Marks)                         | Description                                                                                         |
|------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <b>Workflow File (4 marks)</b>           | .github/workflows/ directory exists; YAML file is valid; trigger set to push on main branch.        |
| <b>Dependency Install Step (3 marks)</b> | Pipeline runs npm install successfully as first step; logged in Actions tab.                        |
| <b>Automated Test Step (5 marks)</b>     | At least one test written and executed in pipeline; pass/fail status visible in GitHub Actions tab. |
| <b>Failed Pipeline Demo (4 marks)</b>    | Intentionally broken test causes pipeline to fail; failure reason visible in logs.                  |
| <b>Fixed Pipeline Demo (4 marks)</b>     | Test fixed; pipeline reruns and passes; green checkmark shown in GitHub Actions.                    |
| <b>Total: 20 marks</b>                   | <b>Passing Threshold: 14 / 20 (70%)</b>                                                             |

| Practical 13: AI API Integration into a Web Application |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|---------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CO/PO                                                   | CO6 / PO3, PO5, PO12                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Objective                                               | To integrate an external AI service into the existing application and understand responsible AI usage.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Prerequisites                                           | <ul style="list-style-type: none"> <li>Practical 6 completed working full-stack Task Management application</li> <li>An active OpenAI API key or Gemini API key (institution-provided or free-tier)</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Coursera Pointer                                        | <b>Theory Concepts (Self-Study Reference)</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|                                                         | No dedicated Coursera module refer to the YouTube video 'Build an AI Image Generator with OpenAI & Node.js' by Traversy Media for general OpenAI API integration patterns in Node.js (adapt the API call pattern to a text-based use case for this practical).                                                                                                                                                                                                                                                                                                                                                                                                             |
|                                                         | <b>Architecture / Diagram</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|                                                         | <pre> graph TD     A[React Frontend] -- "'Generate description' click" --&gt; B[Express Backend /api/ai/generate-description]     B -- "API key stored server-side only (.env)" --&gt; C[OpenAI / Gemini API]     C --&gt; D[Response parsed -&gt; sent back to React -&gt; shown in task form]     E[On API failure: catch error -&gt; return fallback message -&gt; app keeps working]   </pre>                                                                                                                                                                                                                                                                          |
| Diagram                                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Problem Definition</b>                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Problem Statement                                       | <ul style="list-style-type: none"> <li>Add an AI-powered feature to the Task Management application using the OpenAI API or Gemini API.</li> <li>The feature must serve a meaningful purpose for example, auto-generating a task description, suggesting a priority level, or proposing a deadline based on the task title.</li> <li>The API key must be stored and used only on the backend (.env), never exposed to the frontend.</li> <li>Handle API errors gracefully if the AI call fails or times out, the core application must continue to function normally.</li> <li>Display the AI-generated response meaningfully in the React UI, not as raw JSON.</li> </ul> |
| Key Questions / Analysis                                | <ul style="list-style-type: none"> <li>Why must the AI API key be called from the backend rather than directly from the React frontend?</li> <li>What does 'graceful degradation' mean in this context, and why does it matter for a feature that depends on a third-party service outside your control?</li> <li>What are the privacy and governance implications of sending task data to an external AI API, and what kind of data should never be sent this way?</li> </ul>                                                                                                                                                                                             |
| Supplementary Problems                                  | <ul style="list-style-type: none"> <li>Add a loading state on the frontend while waiting for the AI response.</li> <li>Add a simple rate limit (e.g., max 1 AI request per task per minute) to avoid excessive API usage.</li> <li>Display a clear disclaimer in the UI that the suggestion is AI-generated and should be reviewed by the user.</li> </ul>                                                                                                                                                                                                                                                                                                                 |
| Key Skills Addressed                                    | <ul style="list-style-type: none"> <li>Calling an external AI API securely from a Node/Express backend</li> <li>Designing a feature that degrades gracefully when a third-party dependency fails</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

|                                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                | <ul style="list-style-type: none"> <li>• Parsing and displaying AI-generated content meaningfully in a React UI</li> <li>• Reasoning about ethical, privacy, and governance implications of AI integration</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Learning Outcome</b>                        | Students will be able to securely integrate a third-party AI API into a full-stack application, handle failures gracefully, and articulate the responsible-AI considerations involved.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Lab Session Step-by-Step</b>                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Steps</b>                                   | <ol style="list-style-type: none"> <li>1. Install the OpenAI SDK (or use fetch directly for Gemini):<br/> <pre>npm install openai</pre> </li> <li>2. Store the API key in .env (never commit this file):<br/> <pre>OPENAI_API_KEY=your_key_here</pre> </li> <li>3. Create a backend route that accepts a task title and returns a generated description:<br/> <pre>const OpenAI = require('openai'); const client = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });  app.post('/api/ai/generate-description', async (req, res) =&gt; {   try {     const completion = await client.chat.completions.create({       model: 'gpt-4o-mini',       messages: [{ role: 'user', content: `Write a short task description for: \${req.body.title}` } ]     });     res.json({ description: completion.choices[0].message.content });   } catch (err) {     res.status(200).json({ description: '', fallback: true });   } });</pre> </li> <li>4. On the frontend, add a "Generate description" button on the task creation form that calls this endpoint.</li> <li>5. Display the returned description as an editable suggestion the student can accept or modify.</li> <li>6. Test the failure path by temporarily using an invalid API key, and confirm the app shows a fallback message instead of crashing.</li> </ol> |
| <b>Tools / Technology / Resources Required</b> | <ul style="list-style-type: none"> <li>• Node.js (v18+), npm</li> <li>• OpenAI API or Gemini API (institution-provided key)</li> <li>• Express.js, React 18+</li> <li>• Postman (for backend endpoint testing)</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Total Hours</b>                             | <ul style="list-style-type: none"> <li>• Lab Session: 2 hours</li> <li>• Implementation: 1.5 hours</li> <li>• Testing / Modification: 0.5 hours</li> <li>• Total Engagement: 2 hours</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Faculty Teaching Guide</b>                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Guidance Notes</b>                          | <ul style="list-style-type: none"> <li>• Distribute the institution-sponsored API key/access method before the session begins do not let key provisioning eat into lab time.</li> <li>• Emphasize the security point strongly: the API key must never appear in any frontend code or be committed to GitHub.</li> <li>• Use this session to also briefly discuss Responsible AI from the syllabus what data should and should not be sent to a third-party model.</li> <li>• Demonstrate the graceful-degradation failure path live (using a broken key) so students see it's a designed behavior, not a bug.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Common Mistakes</b>                         | <ul style="list-style-type: none"> <li>• Calling the OpenAI/Gemini API directly from the React frontend, exposing the API key publicly in browser network requests.</li> <li>• Not handling API errors, causing the entire task creation flow to break if the AI call fails.</li> <li>• Committing the .env file with the real API key to GitHub.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

|                                                 | <ul style="list-style-type: none"><li>• Treating the AI response as the final answer rather than an editable suggestion, removing user control.</li></ul>                                                                                                                                                                     |                                                                                                        |
|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Troubleshooting Guide                           |                                                                                                                                                                                                                                                                                                                               |                                                                                                        |
| Symptom                                         | Likely Cause                                                                                                                                                                                                                                                                                                                  | Fix                                                                                                    |
| 401 Unauthorized from OpenAI/Gemini             | Invalid or missing API key                                                                                                                                                                                                                                                                                                    | Verify the key in .env is correct and the variable name matches what the code reads                    |
| API key visible in browser DevTools Network tab | AI API called directly from frontend instead of through the backend                                                                                                                                                                                                                                                           | Move the AI API call entirely to an Express route; frontend should only call your own backend endpoint |
| App crashes when AI call fails                  | No try/catch around the API call                                                                                                                                                                                                                                                                                              | Wrap the API call in try/catch and return a fallback response instead of letting the error propagate   |
| Rate limit / quota exceeded error               | Too many requests sent in a short time, especially across a full lab batch                                                                                                                                                                                                                                                    | Add a basic per-task rate limit and inform students in advance to avoid repeated unnecessary calls     |
| Student Assignment / Post Laboratory Work       |                                                                                                                                                                                                                                                                                                                               |                                                                                                        |
| Tasks                                           | <ul style="list-style-type: none"><li>• Review the Theory Concepts section and the AI integration reference video before next week.</li><li>• Add a loading state on the frontend while waiting for the AI response.</li><li>• Add a visible disclaimer noting the content is AI-generated and should be reviewed.</li></ul>  |                                                                                                        |
| GitHub Deliverables                             |                                                                                                                                                                                                                                                                                                                               |                                                                                                        |
| Expected Repository Contents                    | <ul style="list-style-type: none"><li>• Same repository as Practical 6, updated with new commits</li><li>• Working AI-powered feature with graceful failure handling</li><li>• .env excluded via .gitignore; no API key committed anywhere in history</li><li>• At least one commit specifically for AI integration</li></ul> |                                                                                                        |
| Rubrics (Practical Evaluation / Viva)           |                                                                                                                                                                                                                                                                                                                               |                                                                                                        |
| Criteria (Marks)                                | Description                                                                                                                                                                                                                                                                                                                   |                                                                                                        |
| API Integration (5 marks)                       | AI API (OpenAI or Gemini) called correctly from Node backend; API key stored in .env; not exposed in frontend.                                                                                                                                                                                                                |                                                                                                        |
| Feature Meaningfulness (5 marks)                | AI feature serves a real purpose in the app (auto-generate description, prioritize task, suggest deadline); not a raw prompt demo.                                                                                                                                                                                            |                                                                                                        |
| Graceful Degradation (5 marks)                  | App continues to function if AI API fails or times out; error caught and user shown a friendly fallback message.                                                                                                                                                                                                              |                                                                                                        |
| Response Handling (5 marks)                     | AI response parsed correctly; displayed meaningfully in React UI; no raw JSON dumped on screen.                                                                                                                                                                                                                               |                                                                                                        |
| Total: 20 marks                                 | Passing Threshold: 14 / 20 (70%)                                                                                                                                                                                                                                                                                              |                                                                                                        |